MINISTRY OF EDUCATION, CULTURE AND RESEARCH OF REPUBLIC OF MOLDOVA
TECHNICAL UNIVERSITY OF MOLDOVA
FACULTY OF COMPUTERS, INFORMATICS AND MICROELECTRONICS
DEPARTMENT OF SOFTWARE ENGINEERING AND AUTOMATICS

GenDSL - Genetics Problem Solver
Project report

Mentor: prof., Capitan Patricia
Students: Ina Pancenco, FAF-242
Igor Tacu, FAF-242
Maxim Ormanji, FAF-242
Andrei Uzun, FAF-242
Sergiu Gherasim, FAF-242

*By signing the cover page of this report, each student confirms their active involvement in both the development of the project and the writing of the report. Students who did not contribute meaningfully are not permitted to sign.

Submission date: [DD/MM/YYYY]
Chișinău, 2026

ABSTRACT

The project “GenDSL” was developed by the students from the Technical University of Moldova: Maxim Ormanji, Ina Pancenco, Sergiu Gherasim, Igor Tacu and Andrei Uzun.

This project presents GenDSL, a domain-specific language designed to automate the solution of genetics inheritance problems, ranging from basic Mendelian crosses to advanced cases involving linked genes, sex-linked inheritance, and blood group analysis. The language provides a simple, intuitive syntax built specifically for the biology domain, allowing students, teachers, and researchers to describe parents, genes, and inheritance rules using clear, domain-focused commands. GenDSL automates the generation of gametes, Punnett squares, genotype and phenotype distributions, and probability calculations, thereby reducing cognitive overload and eliminating the repetitive, error-prone manual computations that characterize traditional genetics problem-solving.

The language is formally specified through a context-free grammar comprising 44 production rules, 40 non-terminal symbols, and 65+ terminal symbols, organized into seven functional groups covering program structure, declarations, lexical structure, assignments, flow control, expressions, and genetics-specific computations. The implementation uses ANTLR4 to generate a lexer and parser from separate grammar files, following standard compiler design practice with a two-phase approach: lexical analysis (tokenization) followed by syntactic analysis (parsing). A SWOT analysis, business model evaluation, and comparative analysis of existing platforms (StarGenetics, Gizmos, Pearson Genetics AI Solver, GenAlEx) were conducted to position GenDSL within the educational technology landscape.

Describing domain-specific abstraction with open-source transparency and reproducibility, GenDSL offers applications in secondary and higher education, collaborative research, and curriculum integration with learning management systems. Its free, open-source nature and script-based workflow enable educators to share reproducible genetic experiments, while its modular grammar design supports future extension to additional inheritance models.

Keywords: domain-specific language, genetics education, ANTLR4, Mendelian inheritance, grammar specification, lexer, parser, open-source, biology education.

CONTENTS

1 INTRODUCTION 1

2 DOMAIN ANALYSIS 2

2.1 Description and Problem Argumentation 2

2.2 Problem Formulation 3

2.3 Problem Validation 4

2.4 Solution Proposal 5

2.5 Target audience 6

2.6 Value Proposition Canvas 7

2.7 Literature Review 8

3 BUSINESS MODEL AND ANALYSIS 14

3.1 SWOT Analysis and Comparative Analysis of Platforms 15

3.2 System Requirements 16

3.2.1 Functional Requirements 16

3.2.2 Non-Functional Requirements 18

3.3 Business Analysis of the Grammar 19

4 LANGUAGE DESIGN 22

4.1 Grammar Specification in BNF Form 22

4.2 Derivation Tree 26

4.3 Grammar Complexity Metrics 29

4.4 Lexer and Parser Implementation 30

5 CONCLUSIONS 35

BIBLIOGRAPHY 36

APPENDICES 39

Appendix A 39

Appendix B 43

Appendix C 48

Appendix D 49

Appendix E 51

Appendix F 54

Appendix G 63

1 INTRODUCTION

Genetics is a foundational discipline in the biological sciences, underpinning advances in medicine, agriculture, biotechnology, and evolutionary biology. From Gregor Mendel's pioneering experiments with pea plants to the discovery of DNA's double helix structure by Watson and Crick, the field has evolved into a core component of modern scientific education. Today, genetics inheritance problems are a standard part of biology curricula worldwide, requiring students to apply principles such as the laws of segregation and independent assortment, genetic linkage, and probability theory to predict offspring genotypes and phenotypes.

However, as the complexity of these problems increases, from monohybrid to dihybrid, trihybrid, and linked-gene crosses: the manual computation involved grows exponentially. Students who fully understand the underlying biological principles often fail to arrive at correct answers due to arithmetic errors in repetitive, multi-step calculations. This mismatch between conceptual understanding and procedural execution represents a well-documented educational gap in genetics instruction.

Domain-specific languages (DSLs) offer a promising solution to this problem. Unlike general-purpose programming languages, DSLs are tailored to a particular application domain, providing specialized syntax and built-in operations that align directly with domain concepts. Precedents such as Eugene for synthetic biology and GReg for genetic regulatory modeling have demonstrated the viability of this approach in biological contexts.

This report presents GenDSL, a domain-specific language designed to automate the solution of genetics inheritance problems. GenDSL provides an intuitive, biology-focused syntax that allows users to describe parents, genes, and inheritance rules using domain-native commands, and to automatically generate gametes, Punnett squares, genotype and phenotype distributions, and probability calculations. The language is free, open-source, and designed for reproducibility through script-based workflows.

2 DOMAIN ANALYSIS

2.1 Description and Problem Argumentation

Genetics has become one of the most revolutionary scientific disciplines in recent history. It has become the backbone of contemporary medicine, biotechnology, agriculture, and evolutionary biology. From the understanding of genetic diseases to the development of therapies based on genetic information and crop resistance, it has become an essential tool for interpreting how information is passed from one generation to another [14].

The scientific roots of genetics can be traced back to the groundbreaking experiments carried out by Gregor Mendel in the 19th century [15, 16]. Through his systematic study of pea plants, he was able to establish the basic principles of inheritance that are now known as the laws of segregation and independent assortment [4]. Initially, his works were not widely recognized, but in the 20th century, they formed the backbone of classical genetics. Later developments built upon this foundation. Thomas Hunt Morgan showed that genes are located on chromosomes and introduced the idea of genetic linkage by studying fruit flies [17]. The discovery of the double helix structure of DNA by James Watson and Francis Crick in the middle of the 20th century caused another revolution in biology, and genetics became a molecular science [18, 19].

Due to its importance in biology, genetics is included in secondary and advanced education curricula across the world [2, 3]. In most modern education systems, students are presented with a series of inheritance problems from upper secondary lyceum-level biology classes and continue encountering them in university education in life sciences. These problems are not simply theoretical exercises; rather, they help students develop analytical reasoning, probabilistic thinking, and mathematical modeling of biological phenomena [6]. Students are expected to compute probabilities for certain genotypes and phenotypes to appear in offspring, drawing on Mendel's laws of segregation and independent assortment [4], Morgan's law concerning linked genes [10], and other special cases such as blood group inheritance [9, 5].

However, the level of difficulty escalates considerably when more than one pair of genes is involved. A simple cross involving a single pair of genes, like Aa × Aa, can be solved using a straightforward Punnett square [12]. A cross involving two pairs of genes, like AaBb × AaBb, produces many more possible combinations, requiring students to manage multiple variables simultaneously while avoiding arithmetic errors. The situation becomes even more complex when dealing with linked genes and Morgan's Law, where genes do not assort independently and students must understand genetic linkage, recombination, and the frequency of crossing over [8] a conceptual step beyond the application of a simple formula.

As the number of genes involved in a cross increases, the total number of possible gamete combinations grows exponentially. What initially appears to be a structured logical process becomes a combinatorial

expansion problem. A dihybrid cross generates four types of gametes per parent, while trihybrid or linked-gene scenarios introduce even more possible configurations, with each additional gene pair multiplying the number of calculations required [14]. At this stage, the primary challenge is no longer understanding Mendel's principles, but executing a growing number of symbolic and probabilistic operations accurately. The manual construction of Punnett squares becomes increasingly impractical, and even minor arithmetic errors can lead to entirely incorrect genotype or phenotype ratios.

An additional challenge is the sheer volume of theoretical information that must be memorized: definitions, symbolic notation, diagrams, solving methods, terms related to genotype and phenotype, dominance relationships, gamete formation rules, and probability calculations [7]. Unless a student has a strong interest in biology, this accumulation of information can become overwhelming. Solving genetics problems is a complex activity that requires juggling symbolic notation, probability calculations, and biological knowledge all at once. It may start as a straightforward logical exercise, but it quickly becomes mentally demanding as the problems grow more complicated.

2.2 Problem Formulation

The preceding argumentation reveals a central educational problem: genetics instruction often requires students to perform computationally intensive tasks manually, despite the fact that these tasks follow deterministic logical rules [6]. The difficulty lies not in the biological theory itself, but in the mechanical repetition and combinatorial tracking required to apply it. As a result, cognitive resources that could be dedicated to conceptual understanding are instead consumed by procedural calculations.

The problem for many students does not stem from an incorrect comprehension of biological principles, but from the mechanical intricacy of performing repetitive calculations without errors [7]. This points to a clear educational gap: students can comprehend the theory yet fail to arrive at correct results due to avoidable procedural mistakes. Genetics education thereby risks conflating conceptual mastery with arithmetic precision, creating a divide between learners who can manage high symbolic complexity and those who cannot, even when their underlying biological understanding is comparable.

The educational gap can therefore be precisely defined as the mismatch between students' conceptual understanding of inheritance and their ability to correctly execute the combinatorial calculations required to apply that understanding. This formulation points toward a concrete solution direction: tools that automate the procedural and combinatorial aspects of genetics problem-solving would allow students to direct their cognitive effort toward the biological principles themselves, rather than toward error-prone mechanical computation. Bridging this gap through such tools represents a well-defined pedagogical challenge and a clear research objective [11].

2.3 Problem Validation

To complement the literature-based argumentation, an exploratory questionnaire was used to validate whether the identified problem is also visible in the target user population. The questionnaire collected 66 responses and focused on two issues: whether respondents had prior exposure to genetics problems and whether a structured tool would make such problems easier to represent and solve. Although the sample is not intended as a statistically exhaustive study, it is sufficient for validating that the problem is relevant to real learners and that the proposed solution direction is perceived as useful.

The first result shows that the respondent group is highly relevant to the problem domain. A total of 89.4% of participants reported that they had already worked with genetic problems, while only 10.6% had not. This indicates that the majority of answers came from individuals with direct experience of the type of exercises GenDSL is intended to support.

Figure 2.1 – Prior experience with genetic problems among respondents ($n = 66$)

Answer	Share	Percent
Yes		89.4%
No		10.6%

The second result is even more important for product justification. When asked whether a structured tool could help represent genetic patterns more clearly, 74.2% answered "Yes" and the remaining 25.8% answered "Maybe," while no respondents selected "No." In practical terms, this means that all respondents were either directly supportive of such a tool or at least open to its potential value. This strongly supports the feasibility of a DSL-centered solution in an educational setting.

Figure 2.2 – Perceived usefulness of a structured tool for representing genetic patterns ($n = 66$)

Answer	Share	Percent
Yes		74.2%
Maybe		25.8%
No		0.0%

The open-ended responses further clarify the nature of the problem. Several respondents explicitly mentioned unclear task descriptions, difficulty structuring the solution, the time required to write down all intermediate steps, and the frustration of verifying a result after finishing the calculation. Some comments also referred specifically to difficulties with the Punnett square itself. At the same time, a smaller group reported having no major issues, which suggests that the proposed tool should not be framed only as

remediation for struggling learners, but also as an efficiency and verification aid for more advanced users.

Table 2.1 – Recurring themes in the open-ended questionnaire responses

Observed theme	Interpretation for GenDSL design
Unclear or ambiguous descriptions	The tool should provide explicit, domain-specific syntax that reduces interpretation ambiguity.
Too much time spent writing intermediate steps	The language should automate repetitive notation and calculation steps.
Difficulty structuring the task	The grammar should mirror the natural order of genetics problem solving: entities, traits, cross, and result.
Verification is frustrating and time-consuming	The system should make results reproducible and easy to check.
Punnett square-specific difficulties	Core genetics operations should be built in rather than manually reconstructed each time.

Taken together, these findings validate the central problem statement developed earlier in this chapter. The challenge is not simply that genetics is theoretically difficult, but that the procedural representation and verification of genetics problems impose an additional cognitive burden. The survey therefore reinforces the motivation for GenDSL as a structured language that improves clarity, reduces repetitive work, and supports correct reasoning.

2.4 Solution Proposal

The team proposes the creation of a dedicated domain-specific language (DSL) [1, 21] and supporting software tool for solving genetics problems. Domain-specific languages are specialized programming languages tailored to a particular application domain, offering improved expressiveness and ease of use compared to general-purpose languages [20]. Prior work has demonstrated the viability of DSLs in biological domains: Eugene [13] was developed for specifying synthetic biological parts and constraints, while GReg [26] applied DSL principles to modeling genetic regulatory mechanisms.

The tool would provide a simple and intuitive way for students and researchers to describe parents, genes, and inheritance rules using clear, domain-focused commands. This environment would allow users to automatically generate gametes, Punnett squares, genotype and phenotype distributions, blood types and their probabilities [4, 9]. As a result, people who struggle with manual calculations could focus more on understanding genetic concepts rather than on repetitive and error-prone steps [6]. The solution is designed to be easy to learn and suitable even for users with only basic programming experience.

An important advantage of this solution is that it significantly reduces human error when working with multiple genes and complex crosses. By automating the entire calculation process, the DSL helps

users obtain consistent and reproducible results. It also supports learning, because students can clearly see how input data (genes and parents) influence the final outcome [7]. For students who already work with programming and algorithms, the tool connects software engineering skills with real scientific problems in genetics.

This DSL can contribute to a stronger educational and academic environment. Students, teachers, and researchers can share small programs that describe genetic experiments and learning examples, encouraging collaboration and discussion [1]. From an institutional point of view, such a tool supports better teaching of genetics, improves learning efficiency, and helps universities modernize their practical laboratories by introducing automated, technology-driven problem solving.

2.5 Target audience

The stakeholders directly affected by these challenges include high school students struggling with genetics problems, biology teachers tasked with explaining complex concepts, and, in general, individuals involved in biology education. Additionally, developers may be involved in refining the tool, as they contribute to its technical evolution and ensure it aligns with educational needs.

Effective product development hinges on a profound understanding of end users and the specific challenges they encounter in their educational lives. This section delves into the distinctive characteristics, obstacles, and requirements of biology learners and educators who will become the primary users of the GenDSL application. By thoroughly examining their academic environment and daily workflows, the solution can be precisely calibrated to address their most pressing needs, ensuring that the application delivers substantial practical value to its intended audience.

The primary active users of the GenDSL application are biology students and educators. Within this group, three main subgroups can be identified:

- high school students at the lyceum level;
- biology teachers in secondary education;
- researchers or university students engaged in genetic analysis.

While all share a common goal which is improving understanding and efficiency in solving genetics problems, each subgroup faces distinct challenges that the application is designed to resolve.

High School Students serve as the frontline learners in biology classes, particularly in upper-secondary (lyceum) education where genetics problem-solving is most commonly assessed and formalized. These individuals shoulder the considerable responsibility of mastering inheritance patterns, including monohybrid and dihybrid crosses, while dealing with a high volume of combinations, notations, and theoretical information. Their most significant challenge lies in the initial confusion caused by manual calculations that grow error-prone with multiple genes, leading to frustration and difficulty connecting rules like Mendel's laws with practical exercises. GenDSL addresses this fundamental limitation by providing a

simple, intuitive environment built upon domain-specific commands. This enables students to automatically generate gametes, Punnett squares, and genotype/phenotype distributions without repetitive steps, allowing them to focus on conceptual understanding and visualize how input data influences outcomes.

Biology Teachers require rapid capabilities to demonstrate solutions, create batch worksheets, and explain step-by-step processes for common exercises like blood group inheritance or linked genes. The GenDSL platform supports these specialized needs by integrating clear syntax for describing parents, genes, and rules, along with features like explain-mode tracing and export options (PDF/image). This integration allows teachers to model genetic behaviors with unprecedented efficiency. Through adjustable parameters including dominance patterns and recombination, these professionals can swiftly evaluate numerous variants and generate evidence-based teaching materials that carry genuine weight with students and administrators.

Researchers or University Students often encounter significant barriers when attempting to handle advanced or systematic genetic analysis. Established methods demand extensive manual tracking of probabilities and avoidance of computational mistakes, particularly in complex scenarios involving Morgan's law or special cases. GenDSL effectively eliminates these obstacles by offering an intuitive interface for scripting reproducible experiments. The application's advanced features, including strict validation, error-proofing, and support for sharing programs, transform it into both an educational resource and a practical professional tool for those advancing in the field.

In essence, GenDSL empowers biology students and educators across all experience levels and institutional affiliations with a contemporary, precise, and economically viable solution to enhance genetics education. By addressing the specific requirements of each user subgroup while maintaining a unified core functionality, the application positions itself as an indispensable asset in the ongoing effort to improve learning efficiency and problem-solving skills in biology.

2.6 Value Proposition Canvas

The Value Proposition Canvas is a strategic tool that maps the relationship between what the product offers and what the customer needs. It consists of two sides: the Customer Profile (which captures jobs, pains, and gains of the target audience) and the Value Map (which describes the products, pain relievers, and gain creators offered by GenDSL). The canvas below illustrates how GenDSL's features directly address the educational challenges identified in the domain analysis: students' need for error-free genetics problem solving is matched by automated computation, teachers' need for demonstration tools is matched by explain-mode tracing and export capabilities, and researchers' need for reproducible experiments is matched by script-based shareable workflows.

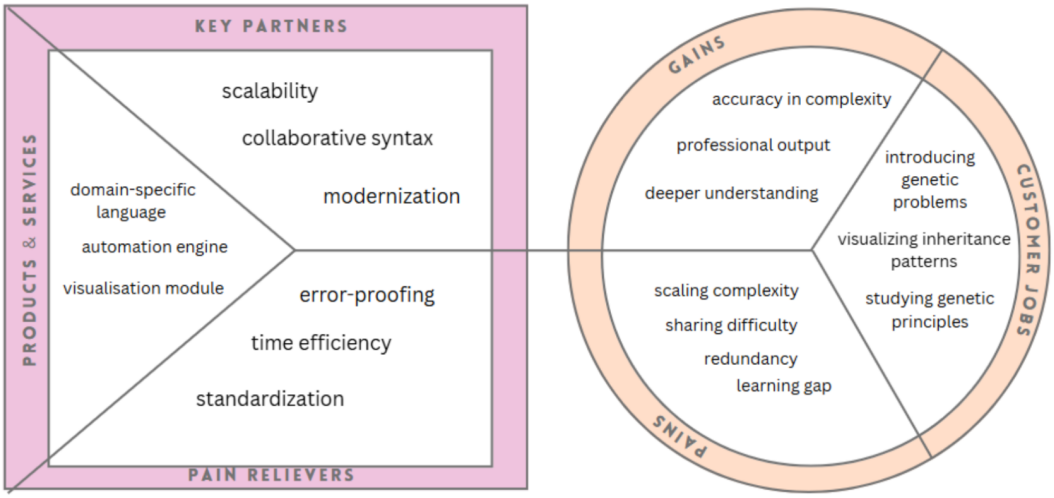

Figure 2.3 – Value proposition canvas

2.7 Literature Review

The literature review was conducted across multiple academic databases, including IEEE Xplore, ACM Digital Library, PubMed/PMC, ERIC, Scopus, and institutional resources from the UTM Scientific Library (Biblioteca Ştiinţifică UTM). Open-access textbook repositories such as OpenStax and NCBI Bookshelf were also consulted. The reviewed sources span three major thematic areas: genetics education research, core genetics theory, and domain-specific language engineering.

Genetics Education and Problem-Solving Difficulties. Research consistently identifies genetics as one of the most difficult topics for students in biology education. Karagöz and Çakır [6] conducted an empirical study showing that students face both conceptual and procedural difficulties when solving genetics problems, with procedural errors (such as incorrect Punnett square construction) being more prevalent than conceptual misunderstandings. Similarly, Mussard and Reiss [34] analyzed examiner reports for 16- to 18-year-olds in England and confirmed that genetics questions are consistently among the worst-performing items, particularly when they require multi-step probabilistic reasoning.

Haskel-Ittah and Yarden [7] further demonstrated that students' conception of genetic phenomena significantly affects their ability to understand underlying mechanisms, suggesting that tools which make the mapping from genotype to phenotype explicit could improve learning outcomes. Avena and Knight [30] found that providing content hints during genetics problem-solving significantly improves student performance, while their follow-up study [31] showed that success rates vary substantially depending on problem type, with linked-gene and multi-allelic problems being the most challenging.

Standardized assessment instruments have been developed to measure genetics understanding. Smith, Wood, and Knight [32] created the Genetics Concept Assessment (GCA), a validated instrument that revealed widespread misconceptions about dominance, probability, and meiosis among undergraduate students. Kalas et al. [33] developed the Meiosis Concept Inventory, identifying persistent difficulties in

understanding chromosomal behavior during cell division, a prerequisite for understanding genetic linkage and recombination.

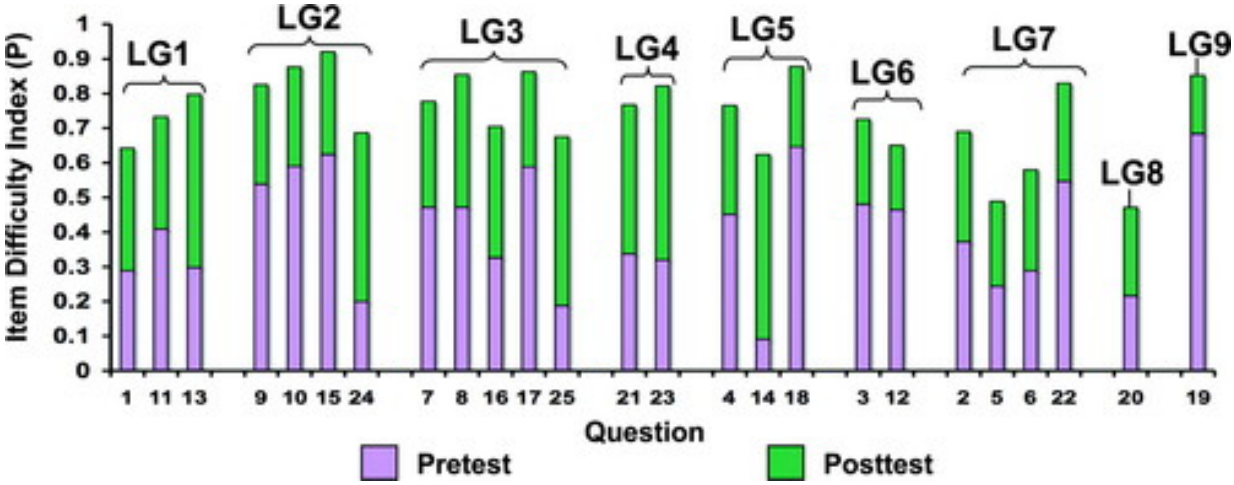

Figure 2.4 – Pretest vs. posttest item difficulty values in the Genetics Concept Assessment, highlighting concept-level learning gains.

Martínez Aznar and Ibáñez Orcajo [35] proposed structured step-by-step methodologies for solving genetics problems, emphasizing that procedural scaffolding can improve success rates. These findings collectively support the rationale for a DSL that automates procedural steps while preserving the conceptual learning objectives.

Core Genetics Theory. The theoretical foundation of the proposed DSL draws from established genetics principles as documented in standard reference works. The OpenStax *Biology 2e* textbook [14] provides comprehensive coverage of Mendelian genetics [4], laws of inheritance including probability rules and epistasis [36], and chromosomal theory with genetic linkage [10]. Fowler, Roush, and Wise [11] extend these principles to cover incomplete dominance, codominance, and multiple alleles, including the ABO blood group system.

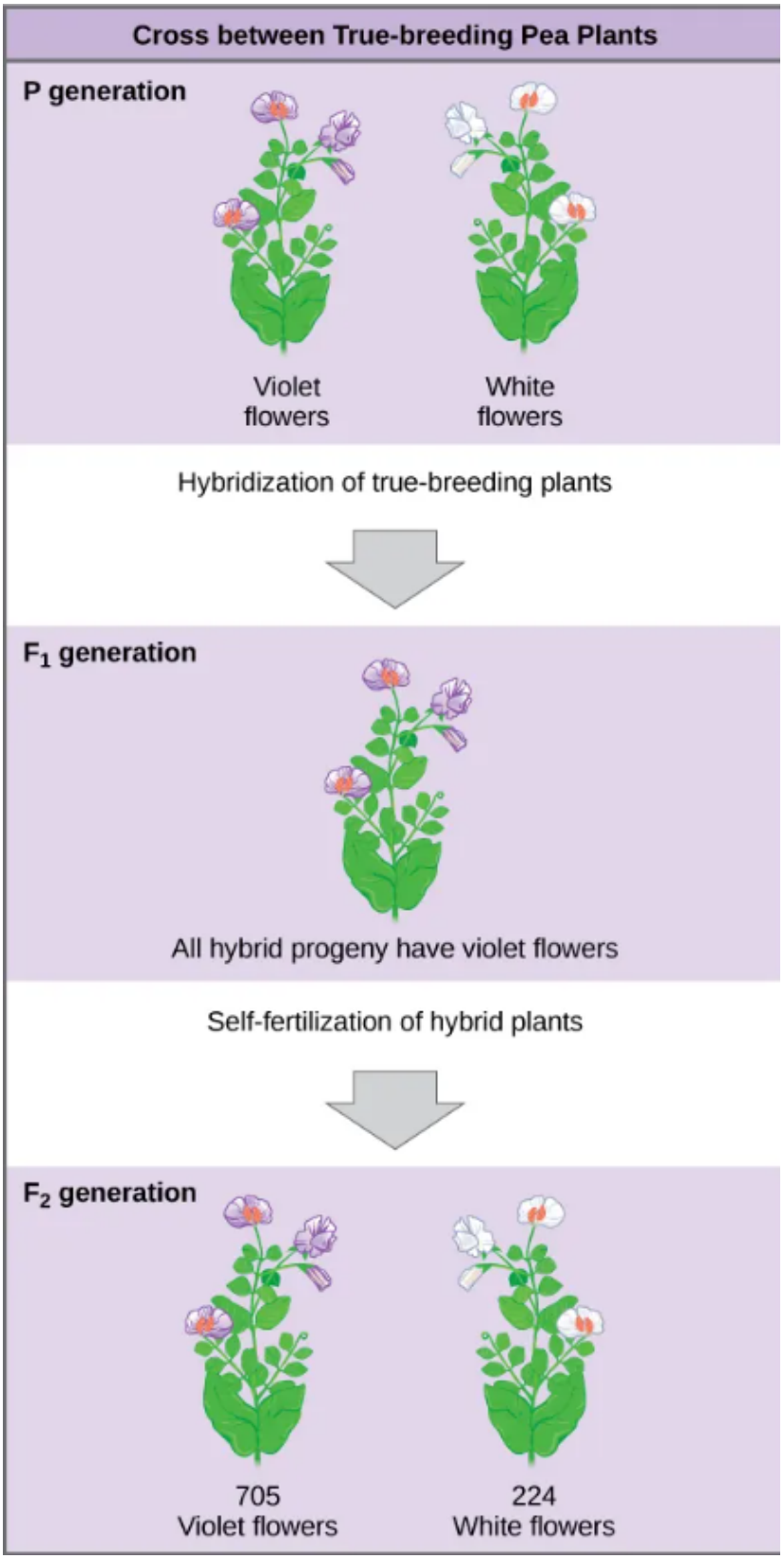

Figure 2.5 – Mendelian inheritance pattern and the characteristic 3:1 segregation in the F2 generation.

Blood group genetics, a common application domain in educational genetics, is authoritatively documented by Dean [9] for the ABO system. The University of Utah Genetic Science Learning Center [5] provides accessible educational materials on genes and blood types, while their Punnett square practice

worksheets [12] illustrate the procedural exercises that the DSL aims to automate.

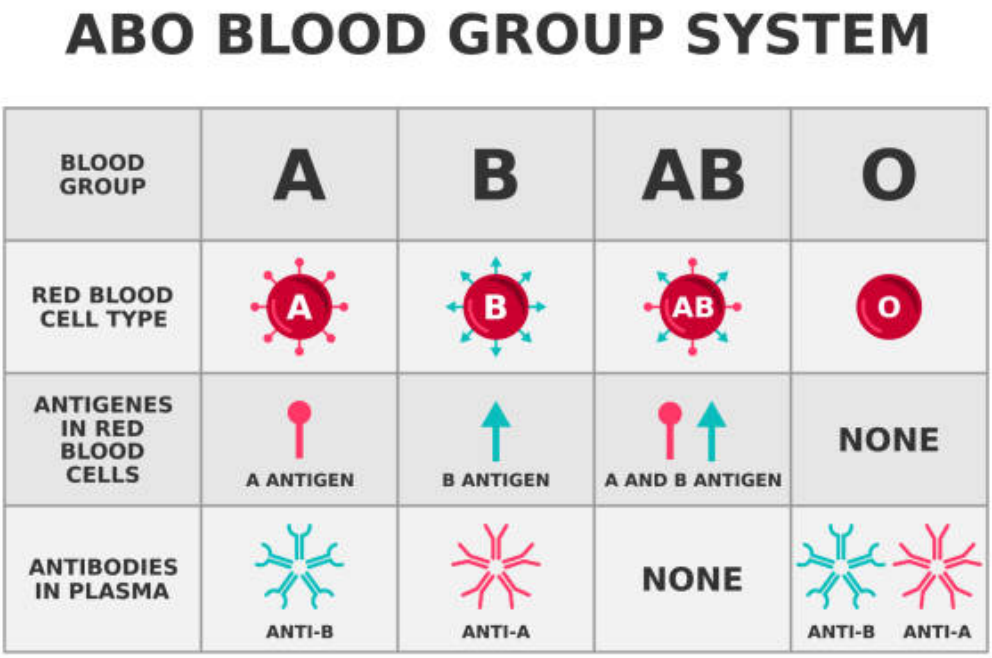

Figure 2.6 – ABO system overview (codominance of A/B alleles and recessive O), used in educational blood-group problem solving.

Meiotic recombination, as reviewed by Hunter [8], provides the biological basis for genetic linkage and crossing over, phenomena that the DSL must model accurately in its linked-gene computation module. The Moldovan biology curriculum [2, 3] aligns with these international standards and provides the specific educational context within which the DSL will be deployed.

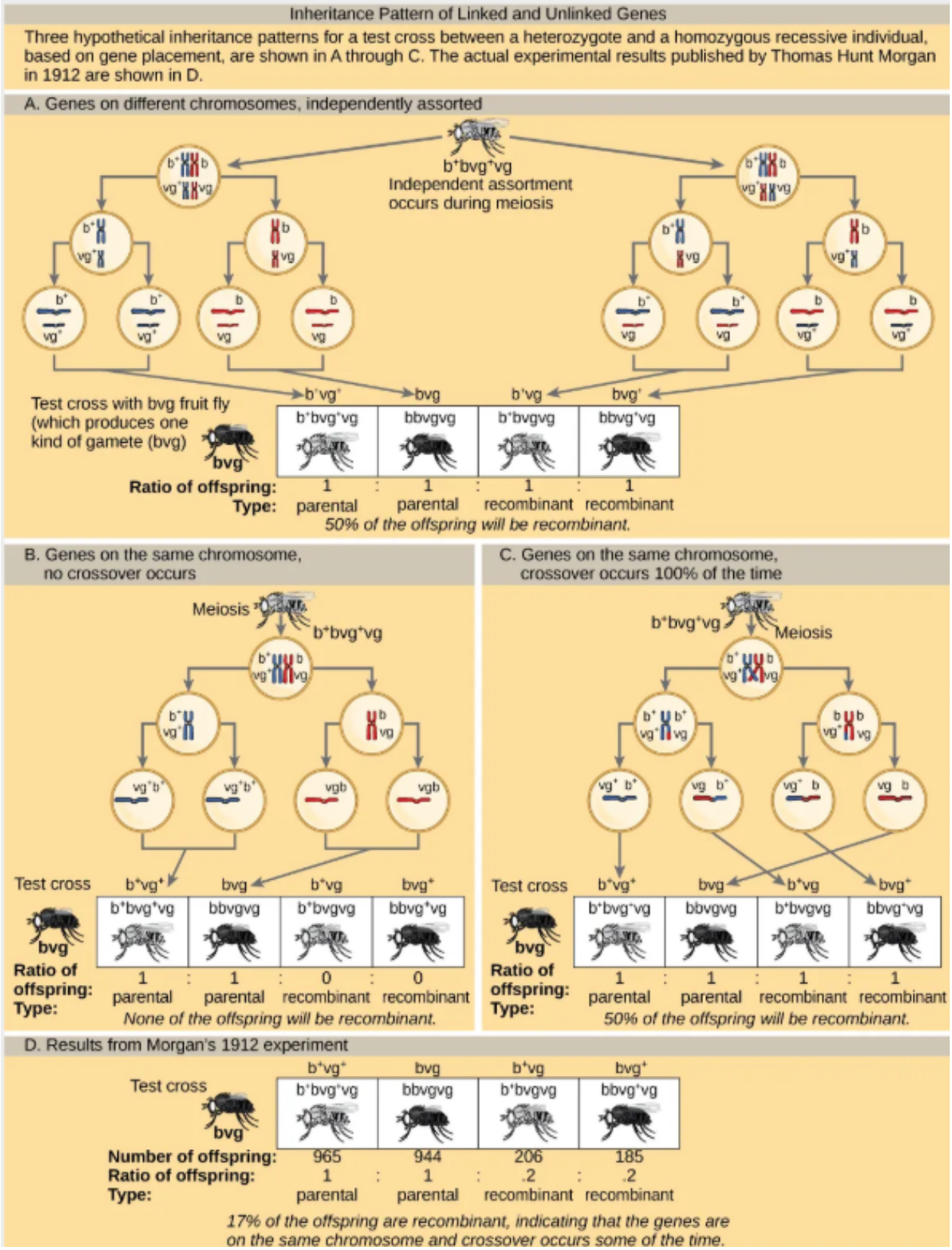

Figure 2.7 – Linked versus unlinked inheritance patterns and recombinant offspring frequency, central to Morgan-law problems.

Historical contributions by Gregor Mendel [15, 16], Thomas Hunt Morgan [17], and Watson and Crick [18, 19] are documented through authoritative biographical sources and provide the intellectual lineage for the genetics concepts encoded in the DSL.

Domain-Specific Language Engineering. The theoretical framework for DSL development is well established in software engineering literature. Van Deursen, Klint, and Visser [20] published an annotated bibliography of DSL research, establishing the key design patterns, advantages (domain-focused abstraction, improved productivity), and risks (language design costs, tooling requirements) that inform DSL development. Mernik, Heering, and Sloane [21] formalized the decision process for when and how to develop domain-specific languages, identifying patterns such as "notation" and "task automation" that directly correspond to the goals of GenDSL.

Fowler's *Domain-Specific Languages* [1] provides practical design methodologies, including the semantic model pattern where the DSL serves as a thin syntactic layer over a well-defined computational core. Parr's *Language Implementation Patterns* [22] offers ANTLR-based implementation techniques for building parsers, abstract syntax trees (ASTs), and interpreters, all of which are directly applicable to the GenDSL implementation.

DSL tooling has been studied from both practical and empirical perspectives. Eysholdt and Behrens [23] demonstrated Xtext as a rapid framework for language development with integrated IDE support. Dantra, Grundy, and Hosking [24] explored visual DSLs using Microsoft DSL tools, highlighting the importance of domain-appropriate notation. Tao et al. [25] empirically studied error messages in data science libraries and proposed improvements directly transferable to DSL diagnostics, an important consideration for a tool aimed at non-programmer users.

Biological Domain-Specific Languages. Several precedents exist for DSLs applied to biological domains. Bilitchenko et al. [13] developed Eugene, a domain-specific language for specifying and constraining synthetic biological parts, devices, and systems. Eugene demonstrated that complex biological constraints can be expressed effectively through domain-focused syntax, validating the DSL approach for biology. Sedlmajer et al. [26] created GReg, a DSL for modeling genetic regulatory mechanisms using Petri net formalisms.

In the broader genomics ecosystem, Krissaane et al. [27] evaluated the Hail framework [28] for genome-wide association studies on cloud platforms, demonstrating how domain-specific operators can enable scalable, reproducible genomic analysis. Ekmekci, McAnany, and Mura [29] introduced Python-based programming for bioscientists, bridging the gap between biological domain knowledge and computational literacy, a goal shared by GenDSL.

These precedents confirm that DSLs for biological applications are both feasible and valuable, while highlighting the importance of domain-appropriate abstraction, user-friendly error messages, and reproducible script-based workflows.

3 BUSINESS MODEL AND ANALYSIS

GenDSL delivers a clear, automated, and error-resistant way to solve genetics inheritance problems, ranging from basic Mendelian crosses to advanced cases involving linked genes and blood groups. Its core value lies in reducing cognitive overload and eliminating repetitive manual calculations, allowing users to focus on conceptual understanding rather than mechanical steps.

The Business Model focuses on complete user freedom without any cost. By providing a free, open-source environment, the project eliminates the ”black box” nature of proprietary software, allowing students to not only see a result but also audit the logic behind it. For schools with limited budgets, this DSL offers high-level bioinformatics capabilities without the burden of licensing fees, ensuring that students' success is determined by their curiosity rather than their financial resources.

Even though the use of the DSL is completely free of charge, the Business Model's focal point is Grants and Funding. By applying for educational technology grants, such as NSF or ERASMUS+, the project can ensure that the DSL continues its development without the risk of running out of financial resources. Moreover, the team will also be accepting sponsorships from biotech companies or universities. This will provide an additional financial source in the case that grant funding is not secured. Another financial source that the project will pursue are professional services, such as paid customization for school or university lab integrations, or custom curriculum development. Not only will this permit the further development of the DSL, but it will also provide the team with a reliable financial source that could be used for further development of the project.

Since the DSL will be free and open source, the project will also be open to contributions. Students, teachers, professors, and enthusiasts all around the world will have the ability to contribute to the DSL's development. A GitHub repository will allow them to fork the project, clone it locally, and open pull requests for changes they have added. The GitHub repository will also include ToDos in the issues tab, which will be the first place contributors will go to when searching for contributions. The inclusion of a clear and comprehensive CONTRIBUTIONS.md file in the root directory of the DSL will likewise be provided. There, all the rules for contributors will be listed.

Distribution will be provided through easy-to-use package managers, such as flatpak, homebrew, pip, npm, apt, as well as via a simple setup file available on the DSL's official website. A hosted “Try it Now” website will also be provided to lower the barrier for non-coders. Being free and open source allows the project to establish a strong presence in the biotech market, as most software is exclusively proprietary. Below is a table of potential competitors:

Table 3.1 – Comparative Analysis of Existing Platforms

Project	Description	Target Audience	Primary Drawback
StarGenetics (MIT)	A virtual "mating" laboratory where students cross organisms (flies, peas) to discover traits.	Undergraduates / High School	High barrier to entry; focused on virtual lab simulation rather than problem-solving.
Gizmos (ExploreLearning)	Interactive web simulations for breeding (Mouse/Chicken genetics) using visual UI.	Grades 3–12	Paid/Proprietary. Limited to specific predefined scenarios; no "custom" coding.
Pearson Genetics AI Solver	An AI-driven tool that provides step-by-step answers to specific textbook questions.	Students	"Cheat" tool rather than a logical tool. Students don't learn the *logic* of the code.
GenAlEx (Excel Add-in)	A tool for population genetic analysis that runs within Microsoft Excel.	Researchers / Grad Students	Extremely outdated UI; requires Excel; steep learning curve for non-data scientists.

3.1 SWOT Analysis and Comparative Analysis of Platforms

A SWOT analysis evaluates the internal strengths and weaknesses of the GenDSL project alongside the external opportunities and threats that may influence its adoption and long-term viability. This strategic assessment helps identify areas where the DSL excels, such as its domain-specific design and open-source nature, as well as potential challenges including competition from general-purpose AI tools and the inherent difficulty of convincing educators to adopt new technologies. Table 3.2 summarizes these factors.

The SWOT analysis reveals that GenDSL's primary competitive advantage lies in its domain-specific design and open-source transparency, which distinguish it from proprietary alternatives. The most significant external threat comes from general-purpose AI tools that can solve basic genetics problems without specialized software. To mitigate this, GenDSL emphasizes reproducibility, curriculum integration, and the ability to model complex multi-gene interactions that conversational AI cannot reliably handle. The opportunities for integration with learning management systems (Moodle, Canvas) and collaborative research workflows provide a clear path toward sustainable adoption.

The detailed project plan (timeline, methodology, resources, and team distribution) is provided in Appendix D.

Table 3.2 – SWOT Analysis

Strengths	**Weaknesses**
Domain-Specific: The syntax is built specifically for biology. Open Source: Transparent, free, and community-auditable. Reproducibility: A script can be saved and shared, unlike a one-off web calculator.	Initial Adoption: Hard to convince busy teachers to learn a "new language," even a simple one. UI/UX Gap: Text-based DSLs can be intimidating for younger high school students compared to visual drag-and-drop. Feature Scope: Manual programming of every edge case (epistasis, linkage) is time-consuming.
Opportunities	**Threats**
Curriculum Integration: Can be bundled as a "computational biology" module in modern science classes. Collaborative Research: Researchers can share DSL scripts to describe experimental setups exactly. API Economy: Can be integrated as a plugin for LMS like Moodle or Canvas.	AI Competition: General AI (like ChatGPT) can solve basic Punnett squares, potentially making specialized tools feel redundant. Tech Burnout: Schools may resist "one more app". Maintenance: Open-source projects require active community management.

3.2 System Requirements

This chapter defines the functional and non-functional requirements of the GenDSL domain-specific language. These requirements emerge directly from the educational gap identified in Chapter 1 and the grammar design decisions presented in Chapter 3: namely, the need for a tool that automates the procedural complexity of genetics problem-solving while remaining accessible to users without a programming background. Functional requirements describe the concrete capabilities the system provides, covering language constructs, genetics-specific computations, control flow, and the underlying lexer/parser infrastructure. Non-functional requirements address the broader quality attributes that govern how the system behaves in practice, including usability, accessibility, reproducibility, extensibility, performance, and maintainability.

3.2.1 Functional Requirements

The functional requirements specify what the system must be able to do in order to fulfill its role as a domain-specific language for genetics education. They cover the complete scope of language constructs, from type declarations and dominance configuration to the nine genetics computation types supported by the grammar, as well as the control flow mechanisms and lexer/parser infrastructure that underpin the entire system.

Language and Grammar.

- The system supports declaration of genetics-specific types: `gene`, `genes`, `parent`, `generation`, `boolean`, `string`, and `number`.
- The system allows dominance relationship configuration via `set dom` constructs, enabling users to specify dominant and recessive allele relationships before performing any cross computation.
- The system supports all nine genetics computation types: cross, find, predict, estimate, infer, probability, linkage, sex-linked inheritance, and blood group analysis.
- The system automatically generates gametes, Punnett squares, genotype and phenotype distributions, and probability calculations from a declarative program description, without requiring the user to perform any manual combinatorial steps.
- The system supports ABO and Rh blood group system analysis, including codominance of A and B alleles and the recessive O allele.
- The system supports Morgan's law linked-gene problems, accepting a recombination frequency as input and producing adjusted offspring distributions that account for genetic linkage and crossing over.

Control Flow and Input/Output.

- The system provides `if`/`elif`/`else`, ternary, `while`, and `for` constructs to enable batch processing and conditional logic within GenDSL programs.
- The system supports `print` statements that accept identifiers, field-qualified identifiers, arithmetic expressions, and genetics event accessors such as `phenotype()` and `genotype()`.
- The system supports single-line comments introduced by `//` and block comments delimited by `/*` and `*/`, both of which are silently discarded and do not affect program execution.

Lexer and Parser.

- The system tokenizes GenDSL source code into 65+ classified token types through a dedicated lexer defined in `ELSDLexer.g4`, recognizing keywords, operators, delimiters, literals, and identifiers.
- The system parses the resulting token stream into a hierarchical parse tree through a dedicated parser defined in `ELSDParser.g4`, implementing all 44 production rules of the formal grammar.
- The system correctly resolves operator precedence, ensuring that multiplication and division are evaluated before addition and subtraction, and handles known grammar ambiguities through ANTLR4 disambiguation rules.

Together, these functional requirements cover the full scope of genetics problem types encountered in secondary and higher education, from basic Mendelian crosses to advanced linked-gene and blood group scenarios, ensuring that every computation a student or educator would need to perform manually is expressible as a declarative GenDSL program.

3.2.2 Non-Functional Requirements

The non-functional requirements govern the broader quality attributes of GenDSL as both an educational tool and an open-source software project. While the functional requirements define what the system computes, the non-functional requirements determine how accessible, reliable, and sustainable the system is for its intended users across different institutional and technical contexts.

Usability.

- The language syntax uses biology-native vocabulary, including keywords such as `cross`, `phenotype`, `genotype`, and `bloodgroup`, so that users with a biology background can read and write programs without prior programming experience.
- The language is designed to be learnable by users with only basic programming experience; advanced features such as linkage, sex-linked inheritance, and blood group analysis are progressively discoverable and are not required for simple monohybrid or dihybrid cross programs.

Accessibility and Distribution.

- The tool is free and open-source, planned for distribution through common package managers including `pip`, `npm`, `apt`, `homebrew`, and `flatpak`, as well as via a downloadable setup file on an official project website.
- A hosted "Try it Now" web interface is planned to allow users to execute GenDSL programs directly in a browser, removing the requirement to install any software locally.

Reproducibility.

- All GenDSL programs are plain-text scripts that can be saved, versioned, and shared, enabling educators and researchers to distribute reproducible genetic experiments alongside their teaching materials or publications.

Extensibility.

- The modular structure of the computation rules allows new genetics problem types to be added to the grammar without modifying the core production rules governing program structure, declarations, or flow control.
- The codebase is publicly hosted on a GitHub repository and includes a `CONTRIBUTIONS.md` file that documents the process for submitting pull requests and adding new features, supporting open community participation.

Performance.

- The system executes genetics computations, including dihybrid and trihybrid crosses, instantaneously upon program invocation, replacing manual workflows that typically require between 15 and 20 minutes of calculation per problem.

Maintainability.

– The lexer and parser are defined in separate grammar files, `ELSDLexer.g4` and `ELSDParser.g4` respectively, allowing each phase to be tested, updated, and extended independently without risk of introducing regressions in the other phase.

The requirements presented in this chapter reflect the dual nature of GenDSL as both an educational tool and a formal language system. The functional requirements cover the genetics problem types encountered across secondary and higher education curricula, from simple Mendelian crosses to advanced linked-gene and blood group scenarios. The non-functional requirements address accessibility, reproducibility, and long-term maintainability, ensuring that the system delivers value not only as a computation engine but as a practical, open, and community-driven resource for genetics education. These requirements directly informed the grammar specification, lexer and parser design, and implementation choices described in the preceding chapters.

3.3 Business Analysis of the Grammar

The GenDSL grammar was designed with a clear business rationale: to maximize educational value while minimizing the learning curve for non-programmer users. This section analyzes the grammar from a business perspective, evaluating how its design choices align with stakeholder needs and market positioning.

Grammar Scope and Feature Coverage. The grammar comprises 44 production rules, 65+ terminal tokens, and 40 non-terminal symbols, providing comprehensive coverage of the genetics problem-solving domain. Table 3.3 maps grammar features to the educational requirements identified in the domain analysis.

Table 3.3 – Grammar Feature-to-Requirement Mapping

Grammar Feature	Rules	Educational Requirement
Type declarations (`gene`, `parent`, `generation`)	R4–R5	Model genetic entities directly
Cross expressions (`cross id x id -> id`)	R28	Automate Punnett square computation
Probability expressions	R32	Compute genotype/phenotype probabilities
Blood group system	R35, R40	ABO/Rh blood type inheritance
Linkage and recombination	R33	Morgan's law linked-gene problems
Sex-linked inheritance	R34	X-linked trait computations
Dominance assignment (`set dom`)	R14	Configure dominance relationships
Inference (`infer parents from`)	R31	Reverse genetics (deduce parents)
Control flow (`if/while/for`)	R16	Batch processing and conditional logic
I/O (`print`)	R44	Display results and intermediate steps

Each grammar feature listed above was derived directly from an identified domain requirement. The `gene`, `parent`, and `generation` type declarations (R4–R5) map to the core entities that appear in every genetics textbook problem, allowing users to describe a cross scenario in the same vocabulary they encounter in lectures. The `cross` computation (R28) directly automates the Punnett square procedure — the most error-prone manual step identified in the literature [6]. The `set dom` assignment (R14) makes dominance relationships an explicit, checkable declaration rather than implicit prior knowledge. The `probability` (R32), `linkage` (R33), `sexlinked` (R34), and `bloodgroup` (R35) computation rules each correspond to a distinct problem category covered in the Moldovan lyceum biology curriculum [2], ensuring that the grammar's expressive power aligns precisely with the breadth of scenarios students are assessed on. In this way, every non-terminal and terminal symbol in the grammar is a structured response to the educational gap identified in Section 1 rather than an arbitrary design choice.

Competitive Differentiation Through Grammar Design. The grammar provides several competitive advantages over existing tools identified in the SWOT analysis:

- **domain-native vocabulary**: keywords like `gene`, `cross`, `phenotype`, `genotype`, `bloodgroup`, and `sexlinked` map directly to biological terminology, eliminating the conceptual translation layer required by general-purpose languages [21];

– **genetics-specific operators**: the cross operator (`x`), the arrow operator (`->`) for offspring designation, and the `ratio` keyword for expressing phenotype ratios are vocabulary choices that mirror the notation students already encounter in textbooks [4];
– **built-in genetics computations**: unlike general-purpose tools that require users to implement probability calculations from scratch, GenDSL provides nine dedicated computation types (R26–R35) that encapsulate the procedural complexity identified as the primary pain point in genetics education [6];
– **declarative problem specification**: users describe *what* they want to compute (e.g., `probability phenotype(child) given genotype(parent)`) rather than *how* to compute it, reducing cognitive load and aligning with the pedagogical goal of focusing on concepts over procedures [7].

Risk Assessment of Grammar Complexity. The grammar's complexity metrics, 44 production rules, 9 levels of recursion depth, and 17 optional elements, present both opportunities and risks:

– **expressiveness vs. learnability**: the high rule count ensures comprehensive domain coverage but may overwhelm novice users, though mitigation is possible since users need only a subset of rules for basic problems, with advanced features introduced progressively;
– **ambiguity handling**: the grammar contains known ambiguities in `binary_expr` and `condition`, which are resolved at the parser level through ANTLR4's disambiguation rules, though clear error messages remain essential for user trust [25];
– **extensibility**: the modular structure of computation rules (R26) allows new genetics problem types to be added without modifying the core grammar, supporting the open-source contribution model described in the business plan.

Return on Investment Analysis. From an educational ROI perspective, the grammar enables significant time savings. A manually solved dihybrid cross with linked genes requires approximately 15–20 minutes of careful calculation [6]. The equivalent GenDSL program requires approximately 5 lines of code and executes instantaneously:

```
gene gA = "Aa";
gene gB = "Bb";
parent p1, p2;
set genotype p1 = gA; set genotype p2 = gB;
cross p1 x p2 -> offspring ratio 9, 3, 3, 1;
```

This represents a reduction from procedural error-prone work to a declarative specification that can be verified, shared, and reused, delivering measurable value to all three user segments identified in the target audience analysis.

4 LANGUAGE DESIGN

4.1 Grammar Specification in BNF Form

The grammar specification defines the complete syntax of the GenDSL language using Backus-Naur Form (BNF) notation. A grammar in this formalism consists of four components: a set of non-terminal symbols representing syntactic categories (such as $\langle declarations \rangle$ or $\langle computations \rangle$), a set of terminal symbols representing concrete tokens that appear in source code (such as keywords, operators, and delimiters), a designated start symbol from which all valid programs are derived, and a set of production rules that define how non-terminals can be rewritten as sequences of terminals and non-terminals.

The GenDSL grammar comprises 44 production rules organized into seven functional groups: program structure (R1–R3), which defines the top-level organization of a GenDSL source file; declarations and types (R4–R7), which introduce domain-specific data types such as `gene`, `parent`, and `generation`; lexical structure (R8–R13), which specifies identifier and literal formation; assignments (R14–R15), which handle field setting and dominance relationships; flow control (R16–R17), which provides conditional branching, ternary expressions, and iteration; expressions and operators (R18–R25), which define arithmetic, Boolean, and comparison operations; computations (R26–R35), which encode the nine genetics-specific operations (cross, find, predict, estimate, infer, probability, linkage, sex-linked inheritance, and blood group analysis); and auxiliary rules (R36–R44), which handle events, alleles, blood systems, labels, generation tags, comments, and I/O.

The GenDSL grammar is formally defined as a context-free grammar $G(L) = \{V_n, V_t, S, P\}$, where V_n is the set of non-terminal symbols, V_t is the set of terminal symbols, S is the starting symbol, and P is the finite set of production rules [22].

Non-terminal symbols ($|V_n| = 40$):

$V_n = \{\langle program \rangle, \langle statements \rangle, \langle statement_list \rangle, \langle declarations \rangle, \langle assignments \rangle, \langle flow_structures \rangle,$
$\langle computations \rangle, \langle io \rangle, \langle type \rangle, \langle id \rangle, \langle id_list \rangle, \langle alphanum \rangle, \langle alpha \rangle, \langle bigalpha \rangle,$
$\langle smallalpha \rangle, \langle digit \rangle, \langle char \rangle, \langle field \rangle, \langle expression \rangle, \langle unary_expr \rangle, \langle binary_expr \rangle,$
$\langle expr_list \rangle, \langle condition \rangle, \langle operator \rangle, \langle number \rangle, \langle string \rangle, \langle event \rangle, \langle event_list \rangle,$
$\langle allele \rangle, \langle allele_list \rangle, \langle bloodsys \rangle, \langle cross_expr \rangle, \langle pred_expr \rangle, \langle find_expr \rangle, \langle est_expr \rangle,$
$\langle infer_expr \rangle, \langle prob_expr \rangle, \langle link_expr \rangle, \langle sex_expr \rangle, \langle blood_expr \rangle, \langle param_list \rangle,$
$\langle label_opt \rangle, \langle gen_tag \rangle, \langle comment \rangle\}$

Terminal symbols ($|V_t| = 65+$):

$V_t = \{$ `; , " ", genes, gene, parent, generation, boolean, string, number,`
`+, -, *, /, =, ->, x, <, <=, >, >=, ==, !=, and, or, not, ?, :,`

`if, then, else, elif, while, do, for, in, end, a...z, A...Z, 0...9,`
`., (, ), [, ], set, dom, label, find, cross, pred, estimate, print, all,`
`infer, parents, from, probability, given, confidence, linkage, recombination,`
`distance,`
`sexlinked, autosomal, bloodgroup, system, ABO, Rh, phenotype, genotype, carries,`
`ratio,`
`true, false, //, /*, */}`

Start symbol: $S = \{\langle program \rangle\}$

Production rules (P, 44 rules):

Program structure (R1–R3):

R1. $\langle program \rangle \rightarrow [\langle comment \rangle]^* \ \langle statement_list \rangle$

R2. $\langle statement_list \rangle \rightarrow \langle statements \rangle \ [\langle statement_list \rangle]$

R3. $\langle statements \rangle \rightarrow \langle declarations \rangle \mid \langle assignments \rangle \mid \langle flow_structures \rangle \mid \langle computations \rangle \mid \langle io \rangle \mid \langle comment \rangle$

Declarations and types (R4–R7):

R4. $\langle declarations \rangle \rightarrow \langle type \rangle \ \langle id_list \rangle \ [= \langle expression \rangle] \ ;$

R5. $\langle type \rangle \rightarrow$ `gene` | `genes` | `parent` | `generation` | `boolean` | `string` | `number`

R6. $\langle id \rangle \rightarrow \langle alpha \rangle \ [\langle alphanum \rangle]^*$

R7. $\langle id_list \rangle \rightarrow \langle id \rangle \ [, \langle id_list \rangle]$

Lexical structure (R8–R13):

R8. $\langle alphanum \rangle \rightarrow \langle alpha \rangle \mid \langle digit \rangle \mid \langle char \rangle$

R9. $\langle char \rangle \rightarrow$ `/` | `?` | `.` | `;` | `_` | `" "`

R10. $\langle alpha \rangle \rightarrow \langle bigalpha \rangle \mid \langle smallalpha \rangle$

R11. $\langle smallalpha \rangle \rightarrow$ `a` | ... | `z`

R12. $\langle bigalpha \rangle \rightarrow$ `A` | ... | `Z`

R13. $\langle digit \rangle \rightarrow$ `0` | ... | `9`

Assignments (R14–R15):

R14. $\langle assignments \rangle \rightarrow$ `set` $[\langle field \rangle] \ \langle id \rangle = \langle expression \rangle \ ;$
| `set` $[\langle field \rangle] \ \langle id \rangle = \langle computations \rangle \ ;$

| `set` [⟨*field*⟩] ⟨*id_list*⟩ = ⟨*expr_list*⟩ ;

| `set dom :` ⟨*bigalpha*⟩ `->` ⟨*smallalpha*⟩ ;

R15. ⟨*field*⟩ → `label` | `dom` | `phenotype` | `genotype` | `codominance`

| `location` | `linked` | `sexlinked` | `autosomal` | `ratio`

Flow control structures (R16–R17):

R16. ⟨*flow_structures*⟩ → `if` ⟨*condition*⟩ `then` ⟨*statement_list*⟩

[`elif` ⟨*condition*⟩ `then` ⟨*statement_list*⟩]*

[`else` ⟨*statement_list*⟩] `end ;`

| ⟨*condition*⟩ `?` ⟨*statements*⟩ `:` ⟨*statements*⟩ `;`

| `while` ⟨*condition*⟩ `do` ⟨*statement_list*⟩ `end ;`

| `for` ⟨*id*⟩ `in` ⟨*expr_list*⟩ `do` ⟨*statement_list*⟩ `end ;`

R17. ⟨*condition*⟩ → ⟨*expression*⟩ ⟨*operator*⟩ ⟨*expression*⟩

| `not` ⟨*condition*⟩

| ⟨*condition*⟩ `and` ⟨*condition*⟩

| ⟨*condition*⟩ `or` ⟨*condition*⟩

| `(` ⟨*condition*⟩ `)`

Expressions (R18–R25):

R18. ⟨*expression*⟩ → ⟨*number*⟩ | ⟨*string*⟩ | `true` | `false` | ⟨*id*⟩ | ⟨*event*⟩

| ⟨*unary_expr*⟩ | ⟨*binary_expr*⟩ | `(`⟨*expression*⟩`)`

R19. ⟨*unary_expr*⟩ → − ⟨*expression*⟩ | `not` ⟨*expression*⟩

R20. ⟨*binary_expr*⟩ → ⟨*expression*⟩ + ⟨*expression*⟩ | ⟨*expression*⟩ − ⟨*expression*⟩

| ⟨*expression*⟩ * ⟨*expression*⟩ | ⟨*expression*⟩ / ⟨*expression*⟩

R21. ⟨*expr_list*⟩ → ⟨*expression*⟩ [, ⟨*expr_list*⟩]

R22. ⟨*number*⟩ → [−] ⟨*digit*⟩$^+$ [. ⟨*digit*⟩$^+$]

R23. ⟨*string*⟩ → `"` ⟨*alphanum*⟩$^+$ `"`

R24. ⟨*operator*⟩ → < | > | <= | >= | == | != | `and` | `or`

R25. ⟨*param_list*⟩ → ⟨*id*⟩ [, ⟨*param_list*⟩] | ⟨*expression*⟩ [, ⟨*param_list*⟩]

Computations (R26–R35):

R26. ⟨*computations*⟩ → ⟨*find_expr*⟩ | ⟨*cross_expr*⟩ | ⟨*pred_expr*⟩ | ⟨*est_expr*⟩

| ⟨*infer_expr*⟩ | ⟨*prob_expr*⟩ | ⟨*link_expr*⟩ | ⟨*sex_expr*⟩ | ⟨*blood_expr*⟩

R27. ⟨*find_expr*⟩ → `find` ⟨*field*⟩ ⟨*id*⟩ [⟨*gen_tag*⟩] ;

R28. $\langle cross_expr \rangle \rightarrow$ `cross` $\langle id \rangle$ `x` $\langle id \rangle$ `->` $\langle id \rangle$ [`ratio` $\langle expr_list \rangle$] ;

R29. $\langle pred_expr \rangle \rightarrow$ `pred` $\langle id_list \rangle$ [$\langle gen_tag \rangle$] ;

R30. $\langle est_expr \rangle \rightarrow$ `estimate` $\langle id \rangle$ $\langle number \rangle$ [`confidence` $\langle number \rangle$] ;

R31. $\langle infer_expr \rangle \rightarrow$ `infer parents from` $\langle id \rangle$ [, $\langle id_list \rangle$] ;
| `infer` $\langle field \rangle$ `from` $\langle id \rangle$ [, $\langle id_list \rangle$] ;

R32. $\langle prob_expr \rangle \rightarrow$ `probability` $\langle event \rangle$;
| `probability` $\langle event \rangle$ `given` $\langle event \rangle$;
| `probability` $\langle event \rangle$ `given` $\langle event_list \rangle$;
| `probability` $\langle event_list \rangle$;

R33. $\langle link_expr \rangle \rightarrow$ `linkage` $\langle id \rangle$, $\langle id \rangle$ `recombination` $\langle number \rangle$;
| `linkage` $\langle id_list \rangle$ `recombination` $\langle number \rangle$ [`distance` $\langle number \rangle$] ;

R34. $\langle sex_expr \rangle \rightarrow$ `sexlinked` $\langle id \rangle$;
| `sexlinked` $\langle id \rangle$ $\langle field \rangle$ $\langle expression \rangle$;

R35. $\langle blood_expr \rangle \rightarrow$ `bloodgroup` $\langle id \rangle$ `system` $\langle bloodsys \rangle$;
| `bloodgroup` $\langle id_list \rangle$ `system` $\langle bloodsys \rangle$ [`phenotype` $\langle expr_list \rangle$] ;

Events, alleles, and auxiliary rules (R36–R44):

R36. $\langle event \rangle \rightarrow$ `phenotype(`$\langle id \rangle$`)` | `genotype(`$\langle id \rangle$`)` | `carries(`$\langle id \rangle$, $\langle allele_list \rangle$`)`

R37. $\langle event_list \rangle \rightarrow \langle event \rangle$ [, $\langle event_list \rangle$]

R38. $\langle allele \rangle \rightarrow \langle id \rangle$

R39. $\langle allele_list \rangle \rightarrow \langle allele \rangle$ [, $\langle allele_list \rangle$]

R40. $\langle bloodsys \rangle \rightarrow$ `ABO` | `Rh`

R41. $\langle label_opt \rangle \rightarrow$ `label` $\langle string \rangle$ | ε

R42. $\langle gen_tag \rangle \rightarrow$ `generation` $\langle number \rangle$ | ε

R43. $\langle comment \rangle \rightarrow$ `//` $\langle alphanum \rangle^*$
| `/*` $\langle alphanum \rangle^*$ `*/`

R44. $\langle io \rangle \rightarrow$ `print` $\langle id \rangle$;
| `print` $\langle field \rangle$ [$\langle id \rangle$ | `all` $\langle expression \rangle$] ;
| `print` $\langle expr_list \rangle$;
| `print` $\langle event_list \rangle$;

4.2 Derivation Tree

A derivation tree (also called a parse tree or concrete syntax tree) is a hierarchical, graphical representation that shows how a given input string is generated by a formal grammar. In a derivation tree, the root node corresponds to the start symbol of the grammar ($\langle program \rangle$ in the case of GenDSL), every internal node represents a non-terminal symbol that was expanded using one of the production rules (R1 through R44), and every leaf node represents a terminal symbol (a concrete token such as a keyword, identifier, operator, or delimiter). Reading the leaves from left to right reconstructs the original input string.

Derivation trees serve several important purposes in language design. First, they verify that a grammar can produce the intended language constructs: if a valid input string cannot be derived from the start symbol through a sequence of rule applications, either the grammar or the input is incorrect. Second, they expose the syntactic structure of a program, making explicit how declarations, assignments, control flow structures, and computation expressions are nested within each other. This structural information is essential for building the abstract syntax tree (AST) that the interpreter or compiler will traverse during semantic analysis and execution. Third, derivation trees can reveal ambiguities in the grammar: if the same input string can be produced by two or more distinct derivation trees, the grammar is ambiguous and may require refactoring or disambiguation rules.

For GenDSL, the derivation tree demonstrates how the 44 production rules interact to parse a complete program. Starting from $\langle program \rangle$, the tree branches through $\langle statement_list \rangle$ and $\langle statements \rangle$ into the five major syntactic categories: $\langle declarations \rangle$ (R4, for defining genes, parents, and generations), $\langle assignments \rangle$ (R14, for setting fields and dominance relationships), $\langle flow_structures \rangle$ (R16, covering if/elif/else, ternary, while, and for constructs), $\langle computations \rangle$ (R26, encompassing cross, probability, linkage, sex-linked, blood group, and inference operations), and $\langle io \rangle$ (R44, for print statements). Each branch further expands through type declarations, identifier lists, expressions, conditions, and domain-specific computation sub-rules until every path terminates in a concrete terminal token from the lexer's 65+ token vocabulary.

Figure 4.1 presents the top-level derivation tree for the GenDSL grammar. The subsequent figures present the derivation subtrees for each of the five major syntactic categories.

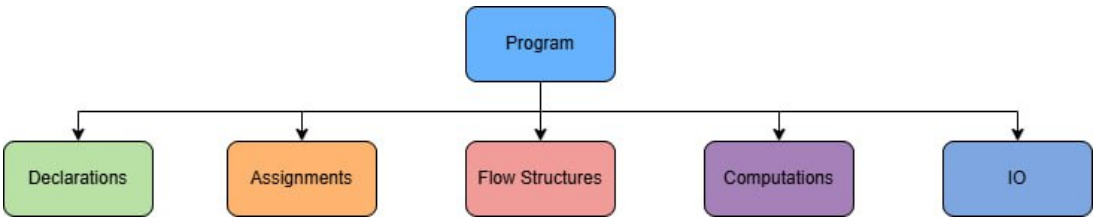

Figure 4.1 – Program structure derivation tree

The top-level derivation tree illustrates how the $\langle program \rangle$ start symbol branches into five major

syntactic categories: Declarations, Assignments, Flow Structures, Computations, and IO. Each category corresponds to a distinct group of production rules in the GenDSL grammar and represents a separate functional domain within the language.

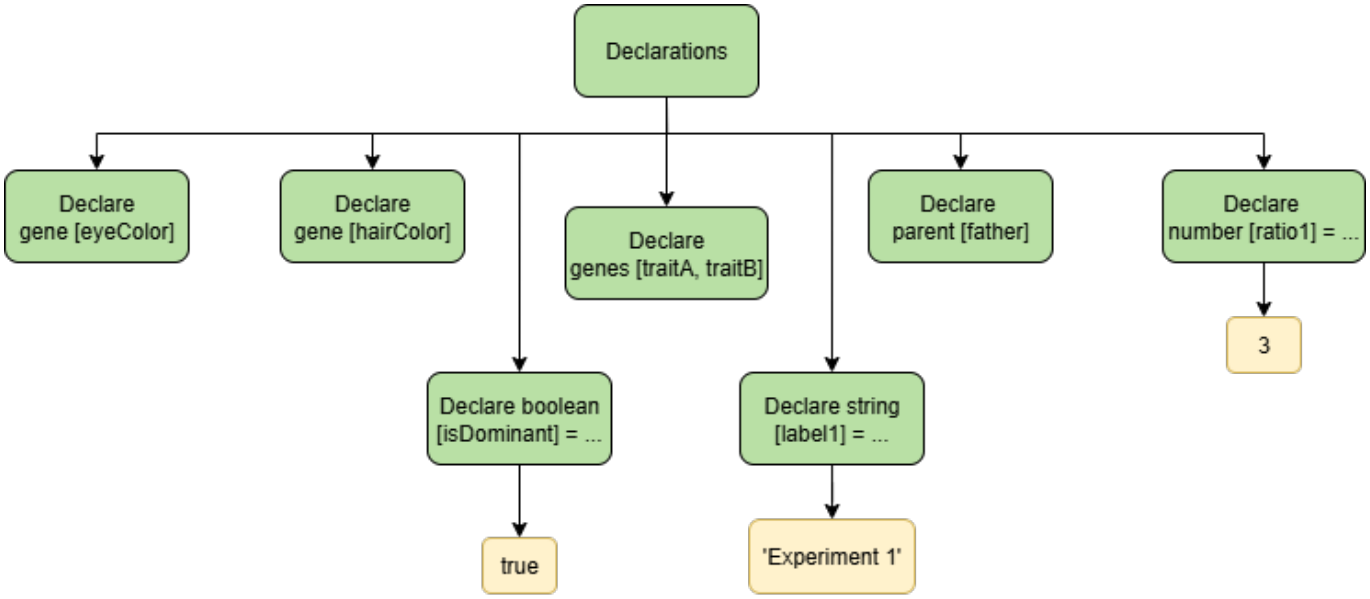

Figure 4.2 – Declarations subtree

The declarations subtree covers production rules R4 through R7 and demonstrates the declaration of genes (single and multi-trait), parents, and primitive types (boolean, string, number) with optional initializers. Each declaration begins with a type keyword followed by an identifier list and an optional assignment expression.

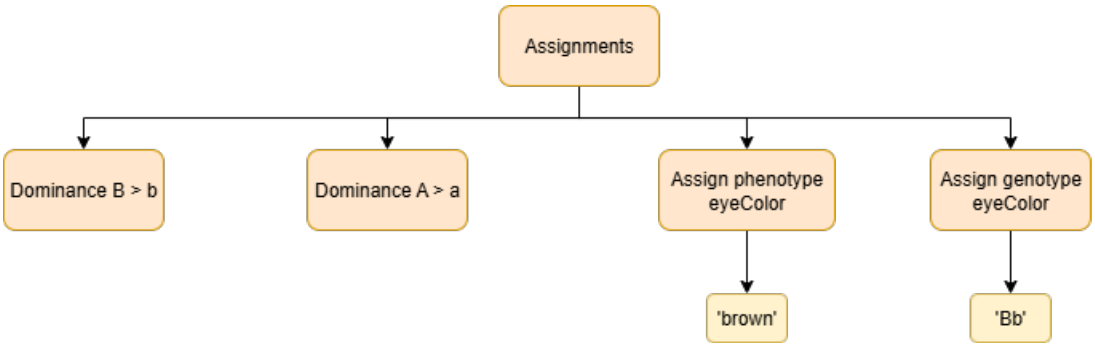

Figure 4.3 – Assignments subtree

The assignments subtree corresponds to production rules R14 and R15, showing how dominance relationships (e.g., B > b, A > a) are established through the `set dom` construct, and how field assignments set phenotype and genotype values for declared entities.

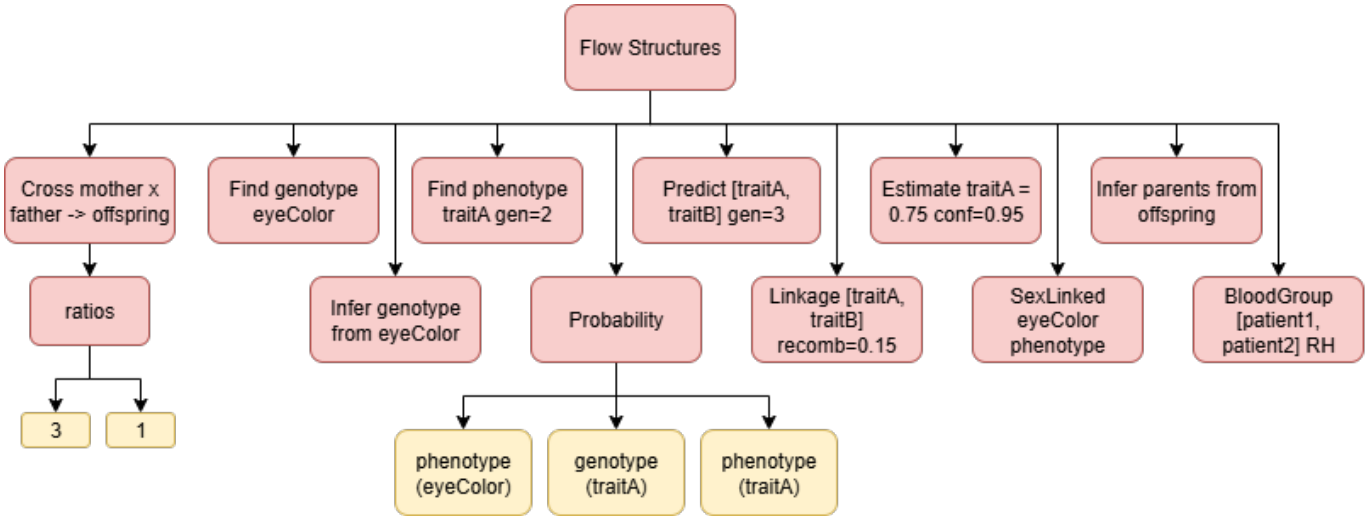

Figure 4.4 – Flow structures subtree

The flow structures subtree illustrates the genetics-specific workflow operations defined in the grammar. These include crossing parents to produce offspring with ratios, finding genotypes and phenotypes, predicting traits across generations with linkage, estimating trait probabilities with confidence intervals, inferring parents from offspring, and performing blood group analysis.

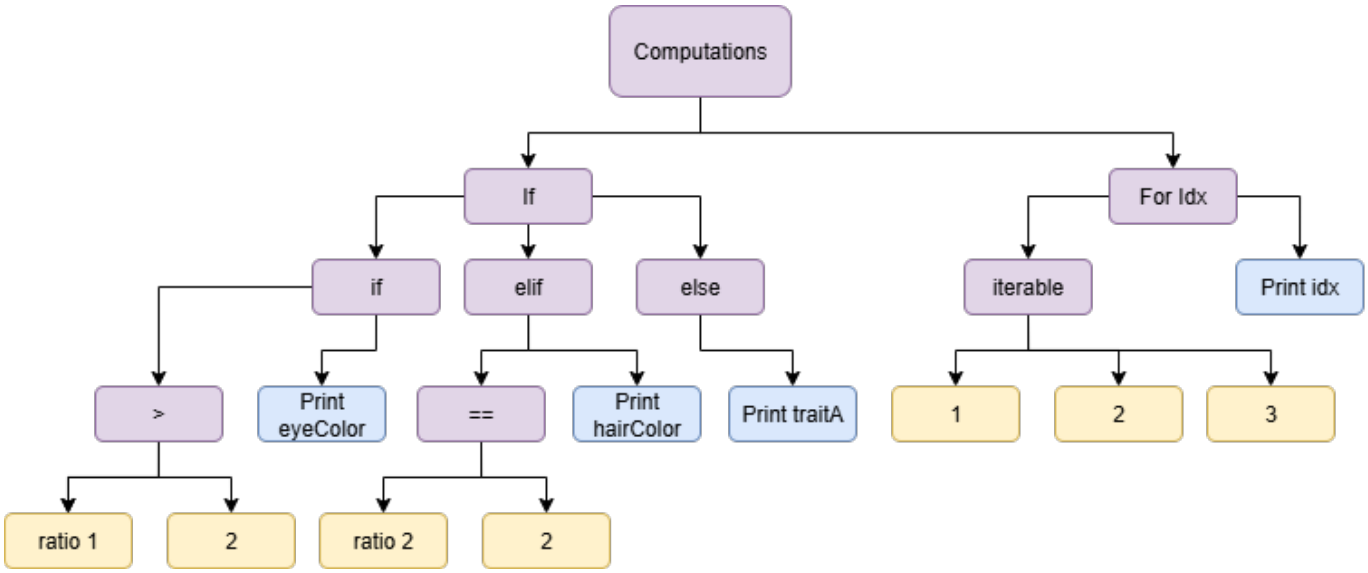

Figure 4.5 – Computations subtree

The computations subtree shows conditional branching through if/elif/else constructs with comparison conditions and corresponding print actions, as well as iterative control flow through a for loop over an iterable sequence. These constructs enable batch processing and conditional logic within GenDSL programs.

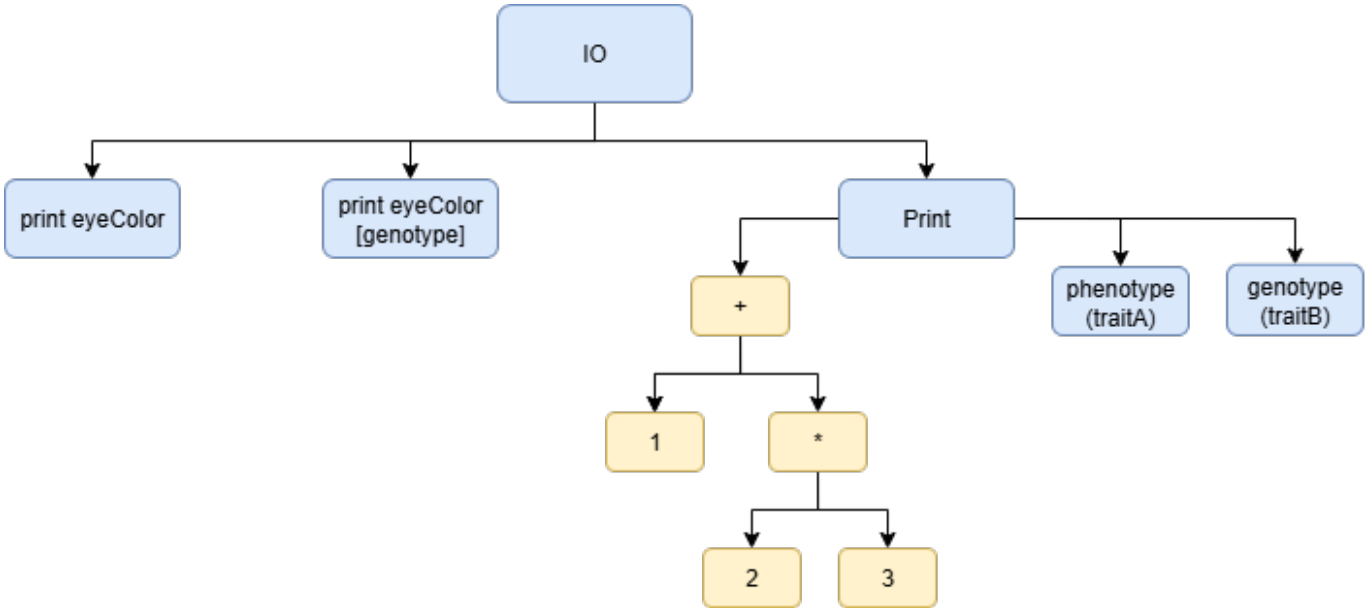

Figure 4.6 – IO subtree

The IO subtree corresponds to production rule R44 and demonstrates the three forms of print statements available in GenDSL: printing an identifier directly, printing a field-qualified identifier (such as a genotype or phenotype), and printing computed expressions alongside event accessors such as `phenotype()` and `genotype()`.

Note: Due to its size, the complete derivation tree — encompassing all subtrees for declarations, assignments, flow structures, computations, and I/O at full nesting depth — is available in full resolution in the project repository.

4.3 Grammar Complexity Metrics

Table 4.1 summarizes the complexity classification of the GenDSL grammar.

Table 4.1 – Grammar Complexity Classification

Metric	Low	Medium	High (GenDSL)
Number of Productions	≤ 10	11–30	44 rules
Recursion Depth	Flat	≤ 3 levels	9 levels
Lexical Rules (Token types)	≤ 10	10–30	65+ tokens
Syntactic Nesting	≤ 2	3–5	9 levels
Optional Elements	Few/none	Some	17 optional slots
Alternative Structures	≤ 2	3–5	Up to 10 per rule
Ambiguity	Unambig.	–	Present

4.4 Lexer and Parser Implementation

GenDSL is implemented using ANTLR4 (ANother Tool for Language Recognition) [22], a powerful parser generator that produces lexers and parsers from grammar specifications. The implementation follows the standard two-phase approach: lexical analysis (tokenization) followed by syntactic analysis (parsing). The lexer and parser are defined in separate grammar files (`ELSDLexer.g4` and `ELSDParser.g4`), allowing independent maintenance and testing of each phase.

Similar to how the Decaf language specification defines its lexical and syntactic structure through a reference grammar and semantic rules, the GenDSL implementation separates concerns into a lexer that handles tokenization (identifying keywords, operators, literals, and identifiers) and a parser that enforces syntactic rules (ensuring tokens appear in valid combinations according to the grammar's production rules). This separation follows standard compiler design practice, where the lexer transforms raw source text into a stream of classified tokens, and the parser consumes that token stream to build a parse tree that reflects the program's hierarchical structure.

Architecture Overview. The GenDSL processing pipeline consists of two sequential stages that transform raw source text into a structured, traversable representation:

$$\textbf{Source Text} \longrightarrow \textbf{Lexer} \longrightarrow \textbf{Token Stream} \longrightarrow \textbf{Parser} \longrightarrow \textbf{Parse Tree}$$

In the first stage, the *lexer* scans the source character-by-character and groups characters into tokens — the smallest meaningful units of the language (keywords such as `gene` or `cross`, identifiers, numeric literals, operators, and delimiters). In the second stage, the *parser* reads the token stream and applies the 44 production rules to construct a hierarchical *parse tree* whose root is $\langle program \rangle$ and whose leaves are the concrete tokens produced by the lexer. The parse tree faithfully encodes the syntactic structure of the input program and serves as the input to subsequent semantic analysis and execution phases.

Lexer. The GenDSL lexer (`ELSDLexer.g4`) defines the complete token vocabulary for the language. It recognizes 65+ token types organized into the following categories:

- **type keywords** (7 tokens): `gene`, `genes`, `parent`, `generation`, `boolean`, `string`, `number`, correspond to the domain-specific data types for modeling genetic entities;
- **flow control keywords** (9 tokens): `if`, `then`, `else`, `elif`, `while`, `do`, `for`, `in`, `end`, provide structured programming constructs;
- **logic keywords** (5 tokens): `and`, `or`, `not`, `true`, `false`, Boolean operations and literals;
- **assignment and field keywords** (11 tokens): `set`, `dom`, `label`, `phenotype`, `genotype`, `codominance`, `location`, `linked`, `sexlinked`, `autosomal`, `ratio`, domain-specific property accessors;
- **computation keywords** (18 tokens): `find`, `cross`, `pred`, `estimate`, `print`, `all`, `infer`, `parents`, `from`, `probability`, `given`, `confidence`, `linkage`, `recombination`,

distance, bloodgroup, system, carries, genetics-specific operations;

- **blood system tokens** (2 tokens): ABO, Rh, blood group system identifiers;
- **operators** (13 tokens): arithmetic (+, −, *, /), assignment (=), arrow (->), cross (x), comparison (<, <=, >, >=, ==, !=), ternary (?, :);
- **delimiters** (7 tokens): ;, ,, ., (,), [,];
- **literals**: NUMBER (integer and floating-point, optionally negative) and STRING_LITERAL (double-quoted strings);
- **identifiers**: ID, letter followed by letters, digits, or underscores.

All GenDSL keywords are lowercase and case-sensitive: gene is a keyword, but Gene or GENE would be treated as identifiers. Keywords and identifiers must be separated by whitespace or a non-alphanumeric token. If a sequence begins with a letter, the longest sequence of letters, digits, and underscores forms a single token. String literals are enclosed in double quotes and may contain any printable character except unescaped double quotes. Numeric literals support both integers and floating-point values, with an optional leading minus sign for negative numbers.

Comments (line comments // and block comments /* ... */) and whitespace are lexed but skipped via the ANTLR -> skip channel directive, keeping the token stream clean for parsing.

The complete lexer grammar is provided in Appendix A.

Parser. The GenDSL parser (ELSDParser.g4) defines the syntactic structure of the language through context-free grammar rules that correspond directly to the BNF production rules R1–R44. The parser uses ANTLR4's tokenVocab option to reference the lexer grammar, ensuring consistency between tokenization and parsing.

Key design decisions in the parser implementation include:

- **labeled alternatives**: ANTLR4's #label syntax is used extensively (e.g., #assignExpr, #ifStatement, #exprMulDiv) to generate distinct visitor/listener methods for each alternative, simplifying subsequent AST traversal and semantic analysis;
- **operator precedence**: expression rules are structured to encode arithmetic precedence: multiplication/division (exprMulDiv) is listed before addition/subtraction (exprAddSub) in the expression rule, leveraging ANTLR4's top-down precedence resolution;
- **modular computation rules**: each genetics computation type (cross, probability, linkage, blood group, etc.) is defined as a separate parser rule, enabling independent testing and future extension.

The complete parser grammar is provided in Appendix B.

Execution Validation Examples.

Simple Example. The minimal program illustrates core declarations, field assignments, cross operations, and basic queries:

```
gene A;
parent mom;
parent dad;
set genotype A = "Aa";
cross mom x dad -> child ratio 1, 1;
find genotype A;
probability genotype(A);
print genotype A;
print "done";
```

Simple Execution. To validate the front-end implementation, the grammar was executed on this program. At the lexical stage, the scanner reported `44` tokens with `0` errors. At the parser stage, ANTLR produced both the LISP-style parse tree and the derived AST:

```
ELSD Parse Result
    File   : examples/simple.elsd
    Tokens : 44
    Errors : 0

Parse Tree excerpt
    (program
        (statementList
            (statement (declaration (type gene) (idList A) ;))
            (statement (assignment set (field genotype) A = (expression "Aa")
             ↪  ;))
            (statement (computation (crossExpr cross mom x dad -> child ratio
                (exprList (expression 1) , (expression 1)) ;)))
            ... ))

AST excerpt
    Program
        Declaration [type=gene, ids=[A]]
        Assign [id=A field=genotype]
            = "Aa"
        Cross [mom x dad -> child]
            ratio: 1, 1
        ...
```

Full execution logs with complete parse trees and ASTs are provided in Appendix E.

Complex Example. The comprehensive program showcases advanced features including dominance relationships, multi-trait declarations, generation tags, probability with conditions, linkage, sex-linked inheritance, blood groups, and control flow:

```
gene eyeColor; genes traitA, traitB; parent mother; parent father;
number ratio1 = 3;
set dom : B -> b;
set phenotype eyeColor = "brown"; set genotype eyeColor = "Bb";
cross mother x father -> offspring ratio 3, 1;
find phenotype traitA generation 2;
pred traitA, traitB generation 3;
probability genotype(traitA) given phenotype(traitA);
linkage traitA, traitB recombination 0.15 distance 20;
sexlinked eyeColor phenotype "red";
bloodgroup patient1, patient2 system Rh phenotype "positive", "negative";
if ratio1 > 2 then print eyeColor; elif ratio1 == 2 then print hairColor;
else print traitA; end;
```

Complex Execution. The comprehensive program parsed with `223` tokens and `0` errors. The grammar successfully handles feature-rich genetics problem descriptions across dominance, probability conditions, linkage analysis, sex-linked traits, blood group systems, and control flow:

```
ELSD Parse Result
    File   : examples/sample.elsd
    Tokens : 223
    Errors : 0

Parse Tree excerpt
    (program
        (statementList
            (statement (declaration (type gene) (idList eyeColor) ;))
            (statement (declaration (type genes) (idList traitA , traitB) ;))
            (statement (assignment set dom : B -> b ;))
            ... ))

AST excerpt
    Program
        Declaration [type=gene, ids=[eyeColor]]
        Declaration [type=genes, ids=[traitA, traitB]]
```

```
Dominance [B -> b]
Assign [id=eyeColor field=phenotype]
Cross [mother x father -> offspring]
...
```

Full execution logs with complete parse trees and ASTs are provided in Appendix F.

Both programs parse without errors, validating the grammar's capacity to handle minimal and feature-rich genetics problem descriptions across the complete feature spectrum.

Together, the lexer and parser grammars define the complete syntactic front-end of GenDSL. The lexer produces a flat stream of 65+ classified tokens from raw source text, and the parser organizes those tokens into a hierarchical parse tree according to the 44 production rules. This parse tree can then be traversed using ANTLR4-generated visitor or listener classes to perform semantic analysis, type checking, or direct interpretation of GenDSL programs.

5 CONCLUSIONS

This report has presented the design and formal specification of GenDSL, a domain-specific language for automating genetics inheritance problem-solving. The work spans domain analysis, business modeling, language design, and lexer/parser implementation.

The domain analysis established a clear educational gap: students understand genetics principles but struggle with the combinatorial complexity of manual calculations as gene count increases. A literature review of over 20 sources confirmed the viability of a DSL approach, and a comparative analysis of existing platforms (StarGenetics, Gizmos, Pearson Genetics AI Solver, GenAlEx) showed that no current tool combines domain-specific syntax, open-source availability, and script-based reproducibility.

The core technical contribution is the formal grammar specification: a context-free grammar with 44 production rules, 40 non-terminal symbols, and 65+ terminal symbols covering program structure, declarations, flow control, expressions, and nine genetics-specific computations. The lexer and parser were implemented using ANTLR4 with separate grammar files (`ELSDLexer.g4` and `ELSDParser.g4`), featuring labeled alternatives for AST generation, structured operator precedence, and modular computation rules.

Future work includes implementing semantic analysis and type checking, developing an interpreter to execute GenDSL programs, building a user-facing interface, and conducting usability testing with biology students and educators.

BIBLIOGRAPHY

[1] Fowler, M. (2010). *Domain-Specific Languages*. Addison-Wesley Professional. Available at: https://www.oreilly.com/library/view/domain-specific-languages/9780132107549/

[2] Ministerul Educației al Republicii Moldova. (n.d.). *Manualul de biologie, clasa a XII-a*. CTICE Moldova. Available at: https://ctice.gov.md/?page_id=447

[3] , XII . (2017). Available at: https://cehov.info/wp-content/uploads/2020/03/XII_-2017.pdf

[4] OpenStax. (2018). Mendel's Experiments and the Laws of Probability. In *Biology 2e*, Chapter 12.1. Available at: https://openstax.org/books/biology-2e/pages/12-1-mendels-experiments-and-the-laws-of-probability

[5] University of Utah Genetic Science Learning Center. (n.d.). Genes and Blood Type. Retrieved February 18, 2026, from https://learn.genetics.utah.edu/content/basics/blood/

[6] Karagöz, M., & Çakır, M. (2011). Problem solving in genetics: Conceptual and procedural difficulties. *Educational Sciences: Theory & Practice*, 11(3), 1668–1674. Available at: https://files.eric.ed.gov/fulltext/EJ936343.pdf

[7] Haskel-Ittah, M., & Yarden, A. (2018). Students' conception of genetic phenomena and its effect on their ability to understand the underlying mechanism. *CBE—Life Sciences Education*, 17(3), ar36. Available at: https://doi.org/10.1187/cbe.18-01-0014

[8] Hunter, N. (2015). Meiotic recombination: The essence of heredity. *Cold Spring Harbor Perspectives in Biology*, 7(12), a016618. Available at: https://doi.org/10.1101/cshperspect.a016618

[9] Dean, L. (2012). ABO blood group. In *Medical Genetics Summaries*. National Center for Biotechnology Information (US). Available at: https://www.ncbi.nlm.nih.gov/books/NBK100894/

[10] Clark, M. A., Douglas, M., & Choi, J. (2018). Chromosomal Theory and Genetic Linkage. In *Biology 2e*, Chapter 13.1. OpenStax. Available at: https://openstax.org/books/biology-2e/pages/13-1-chromosomal-theory-and-genetic-linkage

[11] Fowler, S., Roush, R., & Wise, J. (2013). Extensions of the Laws of Inheritance. In *Concepts of Biology*, Chapter 8.3. OpenStax. Available at: https://openstax.org/books/concepts-biology/pages/8-3-extensions-of-the-laws-of-inheritance

[12] University of Utah Genetic Science Learning Center. (2022, February 28). Practice with Punnett Squares [Worksheet PDF]. Retrieved February 18, 2026, from https://teach.genetics.utah.edu/content/change/pdfs/Condition%20P_Practice%20with%20Punnett%20squares.pdf

[13] Bilitchenko, L., Liu, A., Cheung, S., Weeding, E., Xia, B., Leguia, M., Anderson, J. C., & Densmore, D. (2011). Eugene—A domain specific language for specifying and constraining synthetic biological parts, devices, and systems. *PLOS ONE*, 6(4), e18882. Available at: https://doi.org/10.1371/journal.pone.0018882

[14] OpenStax. (2018). *Biology 2e*. OpenStax. Available at: https://openstax.org/details/books/biology-2e

[15] Encyclopaedia Britannica. (n.d.). Gregor Mendel: Biography, Experiments, & Facts. Retrieved February 18, 2026, from https://www.britannica.com/biography/Gregor-Mendel

[16] Johann Gregor Mendel: Born to be a scientist? (2022). *PMC — PubMed Central*. Available at: https://pmc.ncbi.nlm.nih.gov/articles/PMC9295948/

[17] Nobel Prize Committee. (1933). Nobel Prize in Physiology or Medicine 1933: Thomas Hunt Morgan — Biographical. Available at: https://www.nobelprize.org/prizes/medicine/1933/morgan/biographical/

[18] Biography.com Editors. (n.d.). James D. Watson. Retrieved February 18, 2026, from https://www.biography.com/scientists/james-d-watson

[19] Science History Institute. (n.d.). Francis Crick, Rosalind Franklin, James Watson, and Maurice Wilkins. Retrieved February 18, 2026, from https://www.sciencehistory.org/education/scientific-biographies/francis-crick-rosalind-franklin-james-watson-and-maurice-wilkins/

[20] van Deursen, A., Klint, P., & Visser, J. (2000). Domain-specific languages: An annotated bibliography. *ACM SIGPLAN Notices*, 35(6), 26–36. Available at: https://doi.org/10.1145/352029.352035

[21] Mernik, M., Heering, J., & Sloane, A. M. (2005). When and how to develop domain-specific languages. *ACM Computing Surveys*, 37(4), 316–344. Available at: https://doi.org/10.1145/1118890.1118892

[22] Parr, T. (2009). *Language Implementation Patterns: Create Your Own Domain-Specific and General Programming Languages*. Pragmatic Bookshelf. Available at: https://pragprog.com/titles/tpdsl/language-implementation-patterns/

[23] Eysholdt, M., & Behrens, C. (2010). Xtext: Implement your language faster than the quick and dirty way. In *OOPSLA '10 Companion* (pp. 307–309). ACM. Available at: https://doi.org/10.1145/1869542.1869625

[24] Dantra, R., Grundy, J. C., & Hosking, J. G. (2009). A domain-specific visual language for report writing using Microsoft DSL tools. In *IEEE Symposium on Visual Languages and Human-Centric Computing (VL/HCC)* (pp. 15–22). IEEE. Available at: https://doi.org/10.1109/VLHCC.2009.5295308

[25] Tao, Y., Chen, Z., Liu, Y., Xuan, J., Xu, Z., & Qin, S. (2021). Demystifying "Bad" Error Messages in Data Science Libraries. In *ESEC/FSE 2021*. ACM. Available at: https://doi.org/10.1145/3468264.3468560

[26] Sedlmajer, N., Buchs, D., Hostettler, S., Linard, A., Lopez, E., & Marechal, A. (2011). GReg: A domain specific language for the modeling of genetic regulatory mechanisms. In *BioPPN 2011, CEUR Workshop Proceedings* (Vol. 724, pp. 21–35). Available at: https://ceur-ws.org/Vol-724/paper3.pdf

[27] Krissaane, I., De Niz, C., Gutiérrez-Sacristán, A., et al. (2020). Scalability and cost-effectiveness analysis of whole genome-wide association studies on Google Cloud Platform and Amazon Web Services. *Journal of the American Medical Informatics Association*, 27(9), 1425–1430. Available at: https://doi.org/10.1093/jamia/ocaa068

[28] Hail Team. (2023). Hail (Version 0.2.126) [Computer software]. Zenodo. Available at: https://doi.org/10.5281/zenodo.10055486

[29] Ekmekci, B., McAnany, C. E., & Mura, C. (2016). An Introduction to Programming for Bioscientists: A Python-based Primer. *arXiv preprint arXiv:1605.05419*. Available at: https://doi.org/10.48550/arXiv.1605.05419

[30] Avena, J. S., & Knight, J. K. (2019). Problem Solving in Genetics: Content Hints Can Help. *CBE—Life Sciences Education*, 18(2), ar23. Available at: https://doi.org/10.1187/cbe.18-06-0093

[31] Avena, J. S., McIntosh, B. B., Whitney, O. N., Wiens, A., & Knight, J. K. (2021). Successful Problem Solving in Genetics Varies Based on Question Content. *CBE—Life Sciences Education*, 20(4), ar51. Available at: https://doi.org/10.1187/cbe.21-01-0016

[32] Smith, M. K., Wood, W. B., & Knight, J. K. (2008). The Genetics Concept Assessment: A New Concept Inventory for Gauging Student Understanding of Genetics. *CBE—Life Sciences Education*, 7(4), 422–430. Available at: https://doi.org/10.1187/cbe.08-08-0045

[33] Kalas, P., O'Neill, A., Pollock, C., & Birol, G. (2013). Development of a Meiosis Concept Inventory. *CBE—Life Sciences Education*, 12(4), 655–664. Available at: https://doi.org/10.1187/cbe.12-10-0174

[34] Mussard, J., & Reiss, M. J. (2022). Why is genetics so hard to learn? Insights from examiner reports for 16- to 18-year-olds in England. *School Science Review*, 103(384), 32–40. Available at: https://discovery.ucl.ac.uk/10146651/

[35] Martínez Aznar, M., & Ibáñez Orcajo, T. (2005). Solving problems in genetics. *International Journal of Science Education*, 27(1), 101–121. Available at: https://doi.org/10.1080/09500690410001673801

[36] Clark, M. A., Douglas, M., & Choi, J. (2018). Laws of Inheritance. In *Biology 2e*, Chapter 12.3. OpenStax. Available at: https://openstax.org/books/biology-2e/pages/12-3-laws-of-inheritance

[37] Tacu, I. (2026). *ELSD: A Domain-Specific Language for Genetics Education*. GitHub repository. Available at: https://github.com/igortacu/ELSD

APPENDICES

Appendix A

Complete Lexer Grammar (ELSDLexer.g4)

```
lexer grammar ELSDLexer;

// Keywords: Types
GENE        : 'gene' ;
GENES       : 'genes' ;
PARENT      : 'parent' ;
GENERATION  : 'generation' ;
BOOLEAN     : 'boolean' ;
STRING_TYPE : 'string' ;
NUMBER_TYPE : 'number' ;

// Keywords: Flow Control
IF          : 'if' ;
THEN        : 'then' ;
ELSE        : 'else' ;
ELIF        : 'elif' ;
WHILE       : 'while' ;
DO          : 'do' ;
FOR         : 'for' ;
IN          : 'in' ;
END         : 'end' ;

// Keywords: Logic
AND         : 'and' ;
OR          : 'or' ;
NOT         : 'not' ;
TRUE        : 'true' ;
FALSE       : 'false' ;

// Keywords: Assignment / Fields
SET         : 'set' ;
DOM         : 'dom' ;
LABEL       : 'label' ;
```

```
PHENOTYPE   : 'phenotype' ;
GENOTYPE    : 'genotype' ;
CODOMINANCE : 'codominance' ;
LOCATION    : 'location' ;
LINKED      : 'linked' ;
SEXLINKED   : 'sexlinked' ;
AUTOSOMAL   : 'autosomal' ;
RATIO       : 'ratio' ;

// Keywords: Computations
FIND        : 'find' ;
CROSS       : 'cross' ;
PRED        : 'pred' ;
ESTIMATE    : 'estimate' ;
PRINT       : 'print' ;
ALL         : 'all' ;
INFER       : 'infer' ;
PARENTS     : 'parents' ;
FROM        : 'from' ;
PROBABILITY : 'probability' ;
GIVEN       : 'given' ;
CONFIDENCE  : 'confidence' ;
LINKAGE     : 'linkage' ;
RECOMBINATION : 'recombination' ;
DISTANCE    : 'distance' ;
BLOODGROUP  : 'bloodgroup' ;
SYSTEM      : 'system' ;
CARRIES     : 'carries' ;

// Keywords: Blood Systems
ABO         : 'ABO' ;
RH          : 'Rh' ;

// Operators
PLUS        : '+' ;
MINUS       : '-' ;
```

```
STAR        : '*' ;
SLASH       : '/' ;
ASSIGN      : '=' ;
ARROW       : '->' ;
CROSS_OP    : 'x' ;
LT          : '<' ;
LE          : '<=' ;
GT          : '>' ;
GE          : '>=' ;
EQ          : '==' ;
NEQ         : '!=' ;
QUESTION    : '?' ;
COLON       : ':' ;

// Delimiters
SEMI        : ';' ;
COMMA       : ',' ;
DOT         : '.' ;
LPAREN      : '(' ;
RPAREN      : ')' ;
LBRACK      : '[' ;
RBRACK      : ']' ;

// Literals
NUMBER  : '-'? DIGIT+ ('.' DIGIT+)? ;
STRING_LITERAL : '"' (~["\r\n])* '"' ;

// Identifiers
ID : LETTER (LETTER | DIGIT | '_')* ;

// Comments (skip)
LINE_COMMENT  : '//' ~[\r\n]* -> skip ;
BLOCK_COMMENT : '/*' .*? '*/' -> skip ;

// Whitespace (skip)
WS : [ \t\r\n]+ -> skip ;
```

```
// Fragments
fragment LETTER : [a-zA-Z] ;
fragment DIGIT  : [0-9] ;
```

Appendix B

Complete Parser Grammar (ELSDParser.g4)

```
parser grammar ELSDParser;
options { tokenVocab = ELSDLexer; }

// R1. program
program : statementList EOF ;

// R2. statement_list
statementList : statement+ ;

// R3. statements
statement
    : declaration
    | assignment
    | flowStructure
    | computation
    | io
    ;

// R4-R5. declarations, type
declaration : type idList (ASSIGN expression)? SEMI ;

type
    : GENE | GENES | PARENT | GENERATION
    | BOOLEAN | STRING_TYPE | NUMBER_TYPE
    ;

// R6-R7. id, id_list
idList : ID (COMMA ID)* ;

// R14. assignments
assignment
    : SET field? ID ASSIGN expression SEMI       # assignExpr
    | SET field? ID ASSIGN computation            # assignComputation
    | SET field? idList ASSIGN exprList SEMI      # assignMulti
```

```
    | SET DOM COLON ID ARROW ID SEMI              # assignDominance
    ;

// R15. field
field
    : LABEL | DOM | PHENOTYPE | GENOTYPE | CODOMINANCE
    | LOCATION | LINKED | SEXLINKED | AUTOSOMAL | RATIO
    ;

// R16. flow_structures
flowStructure
    : IF condition THEN statementList
          (ELIF condition THEN statementList)*
          (ELSE statementList)?
      END SEMI                                  # ifStatement
    | condition QUESTION statement COLON
          statement SEMI                        # ternaryStatement
    | WHILE condition DO statementList END SEMI   # whileStatement
    | FOR ID IN exprList DO statementList END SEMI # forStatement
    ;

// R17. condition
condition
    : NOT condition                             # condNot
    | condition AND condition                   # condAnd
    | condition OR condition                    # condOr
    | expression operator expression            # condCompare
    | LPAREN condition RPAREN                   # condParen
    ;

// R24. operator
operator
    : LT | GT | LE | GE | EQ | NEQ | AND | OR
    ;

// R18-R20. expression, unary_expr, binary_expr
```

```
expression
    : MINUS expression                              # exprUnaryMinus
    | NOT expression                                # exprNot
    | expression (STAR | SLASH) expression          # exprMulDiv
    | expression (PLUS | MINUS) expression          # exprAddSub
    | NUMBER                                        # exprNumber
    | STRING_LITERAL                                # exprString
    | TRUE                                          # exprTrue
    | FALSE                                         # exprFalse
    | event                                         # exprEvent
    | ID                                            # exprId
    | LPAREN expression RPAREN                      # exprParen
    ;

// R21. expr_list
exprList : expression (COMMA expression)* ;

// R25. param_list
paramList : (ID | expression) (COMMA (ID | expression))* ;

// R26. computations
computation
    : findExpr | crossExpr | predExpr | estExpr
    | inferExpr | probExpr | linkExpr | sexExpr | bloodExpr
    ;

// R27. find_expr
findExpr : FIND field ID genTag? SEMI ;

// R28. cross_expr
crossExpr : CROSS ID CROSS_OP ID ARROW ID
                (RATIO exprList)? SEMI ;

// R29. pred_expr
predExpr : PRED idList genTag? SEMI ;
```

```
// R30. est_expr
estExpr : ESTIMATE ID NUMBER (CONFIDENCE NUMBER)? SEMI ;

// R31. infer_expr
inferExpr
    : INFER PARENTS FROM ID (COMMA idList)? SEMI  # inferParents
    | INFER field FROM ID (COMMA idList)? SEMI    # inferField
    ;

// R32. prob_expr
probExpr
    : PROBABILITY event SEMI                      # probSimple
    | PROBABILITY event GIVEN event SEMI          # probConditionalSingle
    | PROBABILITY event GIVEN eventList SEMI      # probConditionalMulti
    | PROBABILITY eventList SEMI                  # probMulti
    ;

// R33. link_expr
linkExpr
    : LINKAGE ID COMMA ID RECOMBINATION
          NUMBER SEMI                             # linkPair
    | LINKAGE idList RECOMBINATION NUMBER
          (DISTANCE NUMBER)? SEMI                 # linkMulti
    ;

// R34. sex_expr
sexExpr
    : SEXLINKED ID SEMI                           # sexSimple
    | SEXLINKED ID field expression SEMI          # sexWithField
    ;

// R35. blood_expr
bloodExpr
    : BLOODGROUP ID SYSTEM bloodsys SEMI          # bloodSingle
    | BLOODGROUP idList SYSTEM bloodsys
          (PHENOTYPE exprList)? SEMI              # bloodMulti
```

```
    ;

// R36. event
event
    : PHENOTYPE LPAREN ID RPAREN                  # eventPhenotype
    | GENOTYPE LPAREN ID RPAREN                   # eventGenotype
    | CARRIES LPAREN ID COMMA alleleList RPAREN   # eventCarries
    ;

// R37. event_list
eventList : event (COMMA event)* ;

// R38-R39. allele, allele_list
allele : ID ;
alleleList : allele (COMMA allele)* ;

// R40. bloodsys
bloodsys : ABO | RH ;

// R41. label_opt
labelOpt : LABEL STRING_LITERAL | /* epsilon */ ;

// R42. gen_tag
genTag : GENERATION NUMBER ;

// R44. io
io
    : PRINT ID SEMI                               # printId
    | PRINT field (ID | ALL expression)? SEMI     # printField
    | PRINT exprList SEMI                         # printExprList
    | PRINT eventList SEMI                        # printEventList
    ;
```

Appendix C

Problem Map

This table presents the project problem map in a clearer textual format.

Table 5.1 – Problem Map for GenDSL

Problem Area	Current Situation	Proposed Direction
Manual genetics solving	Multi-gene exercises require repetitive hand calculations and multiple intermediate steps.	Automate routine computations with domain-specific commands and structured execution.
Error rate in exercises	Small arithmetic slips and notation mistakes often lead to incorrect final genotype or phenotype ratios.	Use deterministic parsing, validation, and computation to reduce avoidable human error.
Cognitive overload	Students split attention between inheritance concepts and time-consuming symbolic bookkeeping.	Shift effort toward biological interpretation while the tool handles procedural mechanics.
Teaching workflow	Teachers spend significant time preparing, solving, and checking similar inheritance tasks.	Reuse scripts for faster demonstrations, worksheet generation, and consistent answer verification.
Reproducibility	Handwritten solutions are difficult to compare, review, and reuse across classes.	Represent genetics scenarios as reusable DSL programs that can be shared and rerun.
Accessibility of tools	Existing tools are often too generic, too technical, or poorly aligned with classroom language.	Provide a focused and readable language that follows curriculum terminology and workflows.

Appendix D

Project Plan

The project plan outlines the development roadmap for GenDSL, establishing clear milestones, deliverables, and responsibilities for each phase of the project. The plan is structured around the academic semester schedule at the Technical University of Moldova (UTM) and follows an iterative development methodology that progresses from domain analysis through language design, implementation, and validation. Each milestone builds upon the previous one, ensuring that design decisions made during the grammar specification phase directly inform the lexer and parser implementation, which in turn determines the scope of testing and validation.

Timeline. The project follows a phased development timeline aligned with the academic semester:

Table 5.2 – Project Timeline

Date	Milestone	Deliverable
February 2, 2026	Domain Selection	Genetics education domain chosen
February 7, 2026	Domain Analysis	Problem formulation and solution proposal
February 13, 2026	Literature Review	20+ references across multiple databases
February 20, 2026	Project Plan	Timeline, methodology, resources, team roles
February 27, 2026	Language Syntax Design	Grammar in BNF form (44 production rules)
March 7, 2026	Derivation Trees	Parse trees for all control structures and variable definitions
March 14, 2026	Lexer & Parser	ANTLR4-based implementation (ELSDLexer + ELSDParser)
March 9–21, 2026	Evaluation I	Presentation and demonstration

Methodology. The project follows an iterative development methodology combining elements of domain engineering and language workbench practices [21]:

a) **domain analysis**: identification of genetics problem types, user needs, and computational requirements through literature review and curriculum analysis [6, 2];

b) **language design**: formal grammar specification in BNF notation, informed by domain-specific language design patterns [1, 20], encoding the complete vocabulary of genetics problem-solving;

c) **implementation**: ANTLR4-based lexer and parser development following the parser, AST, interpreter pattern described by Parr [22], tokenizing GenDSL source into 65+ token types across 44 production rules;

d) **testing and validation**: iterative testing with representative genetics problems from curriculum materials [2, 12], covering monohybrid crosses, dihybrid crosses, linked genes, blood groups, and sex-linked inheritance.

Resource Requirements.

- **development tools**: ANTLR4 parser generator, Java/Python runtime, Git version control, Overleaf for documentation;
- **reference materials**: OpenStax Biology 2e [14], Moldovan Grade XII biology textbook [2], genetics practice worksheets [12];
- **hardware**: standard development workstations with internet access;
- **software**: ANTLR4, IDE (IntelliJ IDEA / VS Code with ANTLR4 plugin), LaTeX distribution for documentation.

Team Distribution.

Table 5.3 – Team Roles and Responsibilities

Team Member	Responsibilities
Ina Pancenco	Project coordination, domain analysis, grammar design, documentation
Igor Tacu	Lexer implementation, token specification, testing
Sergiu Gherasim	Parser implementation, AST construction, derivation trees
Andrei Uzun	Literature review, reference management, business analysis
Maxim Ormanji	Validation, test case design, presentation preparation

Appendix E

Sample Program and Full Parser Output (examples/simple.elsd)

Input program (simple.elsd)

```
gene A;
parent mom;
parent dad;
set genotype A = "Aa";
cross mom x dad -> child ratio 1, 1;
find genotype A;
probability genotype(A);
print genotype A;
print "done";
```

Lexer/Parser result

```
Compiling...
Compiled to build/
Running ELSD parser...

ELSD Parse Result
    File   : examples/simple.elsd
    Tokens : 44
    Errors : 0

Parse successful - no syntax errors.

Parse Tree (LISP)

    (program
        (statementList
            (statement
                (declaration
                    (type gene)
                    (idList A) ;))
            (statement
                (declaration
                    (type parent)
                    (idList mom) ;))
```

```
            (statement
                (declaration
                    (type parent)
                    (idList dad) ;))
            (statement
                (assignment set
                    (field genotype) A =
                    (expression "Aa") ;))
            (statement
                (computation
                    (crossExpr cross mom x dad -> child ratio
                        (exprList
                            (expression 1) ,
                            (expression 1)) ;)))
            (statement
                (computation
                    (findExpr find
                        (field genotype) A ;)))
            (statement
                (computation
                    (probExpr probability
                        (event genotype
                            ( A )) ;)))
            (statement
                (io print
                    (field genotype) A ;))
            (statement
                (io print
                    (exprList
                        (expression "done")) ;))) <EOF>)

Abstract Syntax Tree
Program
    Declaration [type=gene, ids=[A]]
    Declaration [type=parent, ids=[mom]]
    Declaration [type=parent, ids=[dad]]
```

```
Assign [id=A field=genotype]
    = "Aa"
Cross [mom x dad -> child]
    ratio: 1, 1
Find [field=genotype, id=A]
Probability
    events: genotype(A)
Print A [field=genotype]
Print "done"
```

Appendix F

Sample Program and Full Parser Output (examples/sample.elsd)

Input program (sample.elsd)

```
// Sample ELSD genetics program
// Demonstrates key features of the DSL

// Declarations
gene eyeColor;
gene hairColor;
genes traitA, traitB;
parent mother;
parent father;
number ratio1 = 3;
boolean isDominant = true;
string label1 = "Experiment 1";

// Dominance assignment
set dom : B -> b;
set dom : A -> a;

// Field assignments
set phenotype eyeColor = "brown";
set genotype eyeColor = "Bb";

// Cross expression
cross mother x father -> offspring ratio 3, 1;

// Find expression
find genotype eyeColor;
find phenotype traitA generation 2;

// Prediction
pred traitA, traitB generation 3;

// Estimation
estimate traitA 0.75 confidence 0.95;
```

```
// Inference
infer parents from offspring;
infer genotype from eyeColor, hairColor;

// Probability
probability phenotype(eyeColor);
probability genotype(traitA) given phenotype(traitA);

// Linkage
linkage traitA, traitB recombination 0.15;
linkage traitA, traitB recombination 0.15 distance 20;

// Sex-linked
sexlinked eyeColor;
sexlinked eyeColor phenotype "red";

// Blood group
bloodgroup patient1 system ABO;
bloodgroup patient1, patient2 system Rh phenotype "positive", "negative";

// Flow control
if ratio1 > 2 then
    print eyeColor;
elif ratio1 == 2 then
    print hairColor;
else
    print traitA;
end;

for idx in 1, 2, 3 do
    print idx;
end;

// I/O
print eyeColor;
```

```
print genotype eyeColor;
print 1 + 2 * 3;
print phenotype(traitA), genotype(traitB);
```

Lexer/Parser result

```
Compiling...
Compiled to build/
Running ELSD parser...

ELSD Parse Result
    File   : examples/sample.elsd
    Tokens : 223
    Errors : 0

Parse successful - no syntax errors.

Parse Tree (LISP)

    (program
        (statementList
            (statement
                (declaration
                    (type gene)
                    (idList eyeColor) ;))
            (statement
                (declaration
                    (type gene)
                    (idList hairColor) ;))
            (statement
                (declaration
                    (type genes)
                    (idList traitA , traitB) ;))
            (statement
                (declaration
                    (type parent)
                    (idList mother) ;))
            (statement
```

```
    (declaration
        (type parent)
        (idList father) ;))
(statement
    (declaration
        (type number)
        (idList ratio1) =
        (expression 3) ;))
(statement
    (declaration
        (type boolean)
        (idList isDominant) =
        (expression true) ;))
(statement
    (declaration
        (type string)
        (idList label1) =
        (expression "Experiment 1") ;))
(statement
    (assignment set dom : B -> b ;))
(statement
    (assignment set dom : A -> a ;))
(statement
    (assignment set
        (field phenotype) eyeColor =
        (expression "brown") ;))
(statement
    (assignment set
        (field genotype) eyeColor =
        (expression "Bb") ;))
(statement
    (computation
        (crossExpr cross mother x father -> offspring ratio
            (exprList
                (expression 3) ,
                (expression 1)) ;)))
```

```
(statement
    (computation
        (findExpr find
            (field genotype) eyeColor ;)))
(statement
    (computation
        (findExpr find
            (field phenotype) traitA
            (genTag generation 2) ;)))
(statement
    (computation
        (predExpr pred
            (idList traitA , traitB)
            (genTag generation 3) ;)))
(statement
    (computation
        (estExpr estimate traitA 0.75 confidence 0.95 ;)))
(statement
    (computation
        (inferExpr infer parents from offspring ;)))
(statement
    (computation
        (inferExpr infer
            (field genotype) from eyeColor ,
            (idList hairColor) ;)))
(statement
    (computation
        (probExpr probability
            (event phenotype
                ( eyeColor )) ;)))
(statement
    (computation
        (probExpr probability
            (event genotype
                ( traitA )) given
            (event phenotype
```

```
                ( traitA )) ;)))
(statement
    (computation
        (linkExpr linkage traitA , traitB recombination 0.15 ;)))
(statement
    (computation
        (linkExpr linkage
            (idList traitA , traitB) recombination 0.15 distance
            ↪  20 ;)))
(statement
    (computation
        (sexExpr sexlinked eyeColor ;)))
(statement
    (computation
        (sexExpr sexlinked eyeColor
            (field phenotype)
            (expression "red") ;)))
(statement
    (computation
        (bloodExpr bloodgroup patient1 system
            (bloodsys ABO) ;)))
(statement
    (computation
        (bloodExpr bloodgroup
            (idList patient1 , patient2) system
            (bloodsys Rh) phenotype
            (exprList
                (expression "positive") ,
                (expression "negative")) ;)))
(statement
    (flowStructure if
        (condition
            (expression ratio1)
            (operator >)
            (expression 2)) then
        (statementList
```

```
            (statement
                (io print eyeColor ;))) elif
        (condition
            (expression ratio1)
            (operator ==)
            (expression 2)) then
        (statementList
            (statement
                (io print hairColor ;))) else
        (statementList
            (statement
                (io print traitA ;))) end ;))
(statement
    (flowStructure for idx in
        (exprList
            (expression 1) ,
            (expression 2) ,
            (expression 3)) do
        (statementList
            (statement
                (io print idx ;))) end ;))
(statement
    (io print eyeColor ;))
(statement
    (io print
        (field genotype) eyeColor ;))
(statement
    (io print
        (exprList
            (expression
                (expression 1) +
                (expression
                    (expression 2) *
                    (expression 3)))) ;))
(statement
    (io print
```

```
                    (exprList
                        (expression
                            (event phenotype
                                ( traitA ))) ,
                        (expression
                            (event genotype
                                ( traitB )))) ;))) <EOF>)

Abstract Syntax Tree
Program
    Declaration [type=gene, ids=[eyeColor]]
    Declaration [type=gene, ids=[hairColor]]
    Declaration [type=genes, ids=[traitA, traitB]]
    Declaration [type=parent, ids=[mother]]
    Declaration [type=parent, ids=[father]]
    Declaration [type=number, ids=[ratio1] = 3]
    Declaration [type=boolean, ids=[isDominant] = true]
    Declaration [type=string, ids=[label1] = "Experiment 1"]
    Dominance [B -> b]
    Dominance [A -> a]
    Assign [id=eyeColor field=phenotype]
        = "brown"
    Assign [id=eyeColor field=genotype]
        = "Bb"
    Cross [mother x father -> offspring]
        ratio: 3, 1
    Find [field=genotype, id=eyeColor]
    Find [field=phenotype, id=traitA gen=2]
    Pred [ids=[traitA, traitB] gen=3]
    Estimate [id=traitA, value=0.75 confidence=0.95]
    Infer parents from offspring
    Infer [field=genotype] from eyeColor, [hairColor]
    Probability
        events: phenotype(eyeColor)
    Probability
        events: genotype(traitA)
```

```
    given:  phenotype(traitA)
Linkage [ids=[traitA, traitB], recomb=0.15]
Linkage [ids=[traitA, traitB], recomb=0.15 distance=20]
SexLinked [id=eyeColor]
SexLinked [id=eyeColor, field=phenotype, value="red"]
BloodGroup [ids=[patient1], system=ABO]
BloodGroup [ids=[patient1, patient2], system=Rh]
    phenotypes: "positive", "negative"
If (ratio1 > 2)
    Print eyeColor
Elif (ratio1 == 2)
    Print hairColor
Else
    Print traitA
For idx in [1, 2, 3]
    Print idx
Print eyeColor
Print eyeColor [field=genotype]
Print (1 + (2 * 3))
Print Event [phenotype(traitA)], Event [genotype(traitB)]
```

Appendix G

Team Contract

Project: GenDSL – Domain-Specific Language for Genetics Inheritance Problems **Institution:** Technical University of Moldova

1. Team Members

The following students are members of the GenDSL development team:

- Maxim Ormanji
- Ina Pancenco
- Sergiu Gherasim
- Igor Tacu
- Andrei Uzun

All members agree to contribute actively to the successful development and completion of the GenDSL project.

2. Project Overview

GenDSL is a domain-specific language designed to automate the solution of genetics inheritance problems. The language enables users to describe parents, genes, and inheritance rules using a clear syntax tailored for the biology domain.

The system automatically generates:

- Gametes
- Punnett squares
- Genotype distributions
- Phenotype distributions
- Probability calculations

The language is formally specified using a context-free grammar and implemented using ANTLR4, following a two-phase compilation process:

1. Lexical analysis (tokenization)
2. Syntactic analysis (parsing)

The project also includes a SWOT analysis, business model evaluation, and comparative analysis of existing educational platforms.

3. Roles and Responsibilities

Each team member agrees to contribute to the project according to their skills and assigned responsibilities.

Possible responsibilities include:

- **Grammar Design** – defining production rules and language structure

- **ANTLR Implementation** – lexer and parser development
- **Testing and Debugging** – validating DSL scripts and outputs
- **Documentation** – writing technical explanations and reports
- **Research and Analysis** – SWOT analysis and comparative platform study

All members are expected to support each other and assist in tasks when necessary.

4. Communication

The team agrees to:

- Communicate regularly regarding progress and issues.
- Inform the group if they are unable to complete a task on time.
- Participate in team discussions and decision-making.
- Communication may occur through:
- Messaging platforms (e.g., Telegram, Discord, or similar)
- In-person meetings
- Online calls when necessary.

5. Work Expectations

All team members agree to:

1. Complete assigned tasks by agreed deadlines.
2. Produce work of reasonable quality.
3. Share progress updates with the team.
4. Respect the time and contributions of other members.
5. Work should be distributed as evenly as possible among team members.

6. Conflict Resolution

If disagreements occur:

1. The team will first attempt to resolve the issue through discussion.
2. If the issue cannot be resolved internally, the team may consult the project supervisor for guidance.

7. Contribution and Accountability

Each member is responsible for contributing meaningfully to the project.

If a team member repeatedly fails to participate or complete assigned tasks, the issue may be documented and reported to the mentor.

8. Agreement

By participating in the GenDSL project, all team members agree to follow the expectations outlined in this contract and work collaboratively toward the successful completion of the project.

Signatures:

Maxim Ormanji _____

Ina Pancenco _____

Sergiu Gherasim _____

Igor Tacu _____

Andrei Uzun _____

Date: 17.03